

UNIVERSITÀ
DI TORINO

Università degli Studi di Torino

Corso di Laurea Triennale in Informatica

Stima della configurazione articolare della mano per la valutazione dell'età ossea pediatrica da immagini radiografiche

Tesi di Laurea

Relatore
Marco Grangetto

Correlatore
Michele Cannito

Candidato
Daniele Giacomelli

Anno Accademico 2025/2026

DICHIARAZIONE DI ORIGINALITÀ

Dichiaro di essere responsabile del contenuto dell'elaborato che presento al fine del conseguimento del titolo, di non avere plagiato in tutto o in parte il lavoro prodotto da altri e di aver citato le fonti originali in modo congruente alle normative vigenti in materia di plagio e di diritto d'autore. Sono inoltre consapevole che nel caso la mia dichiarazione risultasse mendace, potrei incorrere nelle sanzioni previste dalla legge e la mia ammissione alla prova finale potrebbe essere negata.

Indice

Introduzione **iii**

1 Background e stato dell'arte **1**

1.1 Fondamenti di reti neurali 1

1.1.1 Il problema della classificazione di immagini 1

1.1.2 Dal classificatore lineare alle reti neurali 2

1.1.3 L'addestramento di una rete neurale 2

1.1.4 Reti neurali convoluzionali 3

1.1.5 Transfer learning 4

1.2 Approcci automatici alla stima dell'età ossea 4

1.3 Il paradigma BoNet 4

1.4 Architetture di riferimento 5

1.4.1 Hand detection: Faster R-CNN e ResNet 5

1.4.2 Keypoint estimation: Simple Baselines e Keypoint R-CNN 6

1.4.3 Evaluation metrics 7

1.5 Dataset di riferimento 9

2 Materiali e metodi **11**

2.1 Dataset RHPE 11

2.1.1 Analisi esplorativa 11

2.2 Hand Detection 12

2.2.1 Campionamento stratificato 13

2.2.2 Collegamento tra annotazioni CSV e JSON 13

2.2.3 Iperparametri 13

2.2.4 Ciclo di addestramento 13
2.3 Keypoint Estimation 14
2.3.1 Quattro varianti di modello 15
2.3.2 Campionamento e iperparametri 17
2.3.3 Ciclo di addestramento 18
2.3.4 Estensione alla mano destra tramite mirroring 19
2.3.5 Esplorazione su entrambe le mani: metodologia 19
2.4 Metriche di valutazione 23
2.4.1 Sigma OKS 23

3 Risultati sperimentali **25**
3.1 Hand Detection 25
3.2 Keypoint Estimation 26
3.2.1 Retraining di `gt_crop` sul dataset completo 28
3.2.2 Retraining di `mirrored_full` sul dataset completo . . 30
3.3 Esplorazione su entrambe le mani: risultati 32
3.4 Sintesi dei risultati 35

4 Discussione **37**
4.1 Confronto con il paper di riferimento 37
4.2 Osservazioni metodologiche 39
4.3 Limiti del lavoro 41

5 Conclusioni e sviluppi futuri **43**
5.1 Riepilogo dei contributi 43
5.2 Sviluppi futuri 43

Bibliografia **45**

Introduzione

La stima dell'età ossea è una procedura radiologica che confronta il grado di maturazione scheletrica della mano di un paziente pediatrico, osservabile da una radiografia, con la sua età anagrafica. La discrepanza tra le due età è un indicatore clinico utilizzato nella diagnosi di disturbi della crescita e di patologie endocrinologiche, nel monitoraggio di terapie ormonali e, in ambito medico-legale, nella stima dell'età di individui privi di documentazione anagrafica affidabile.

I metodi clinici di riferimento sono il metodo Greulich-Pyle (G&P) [3], che confronta la radiografia con un atlante di immagini rappresentative per ciascuna età, e il metodo Tanner-Whitehouse (TW2) [14], che assegna un punteggio di maturazione a un insieme di regioni ossee della mano analizzate singolarmente. Entrambi richiedono la valutazione soggettiva di un radiologo esperto e presentano una variabilità inter- e intra-osservatore non trascurabile, oltre a tempi di refertazione non immediati.

Questi limiti motivano l'automazione della stima dell'età ossea tramite tecniche di deep learning, con l'obiettivo di fornire una stima oggettiva, riproducibile e rapida a partire dalla sola immagine radiografica.

Obiettivi della tesi

Questo lavoro si propone di replicare ed estendere l'approccio proposto nel paper BoNet [1] (Sez. 1.3), che affronta la stima dell'età ossea tramite un pipeline in tre stadi: hand detection, keypoint estimation e bone age assessment. Il lavoro è condotto sul dataset RHPE, utilizzato dagli stessi autori del paper di riferimento.

Poiché la repository ufficiale del paper non include pesi pre-addestrati, l'obiettivo pratico è addestrare da zero i modelli dei primi due stadi del pipeline e confrontare i risultati ottenuti con quelli riportati dagli autori. A partire da questa replica, il lavoro introduce un'estensione originale non trattata nel paper: la predizione dei keypoint su entrambe le mani presenti in una radiografia, nonostante le annotazioni RHPE coprano esclusivamente

la mano sinistra.

Struttura del documento

Il Capitolo 1 fornisce i concetti di base di reti neurali e apprendimento profondo necessari a comprendere il resto del lavoro, inquadra il problema della stima dell'età ossea nel contesto della letteratura e presenta il paradigma BoNet e le architetture di riferimento utilizzate. Il Capitolo 2 descrive il dataset, le scelte metodologiche e gli iperparametri adottati. Il Capitolo 3 riporta i risultati sperimentali. Il Capitolo 4 discute tali risultati e ne evidenzia i limiti. Il Capitolo 5 riassume i contributi e delinea gli sviluppi futuri.

Capitolo 1

Background e stato dell'arte

1.1 Fondamenti di reti neurali

Tutti i modelli utilizzati in questo lavoro sono reti neurali convoluzionali. Questa sezione ne introduce il funzionamento, seguendo l'impostazione di [2], partendo dal problema di base della visione artificiale, la classificazione di immagini, per arrivare agli elementi che ricorrono nei capitoli successivi: le architetture convoluzionali, il modo in cui vengono addestrate e la pratica di riutilizzare modelli già addestrati su altri dataset.

1.1.1 Il problema della classificazione di immagini

Il compito di riferimento della visione artificiale è la classificazione di immagini: dato un insieme prestabilito di categorie possibili e un'immagine in ingresso, assegnare a quest'ultima la categoria corretta. Per un calcolatore l'immagine non è altro che un tensore di numeri interi, tipicamente compresi tra 0 e 255, che rappresentano l'intensità dei singoli pixel sui tre canali di colore: tra questa rappresentazione numerica e il concetto che l'immagine raffigura non esiste alcuna corrispondenza diretta, una distanza nota come *semantic gap*, che rende impraticabile scrivere a mano una procedura di riconoscimento.

L'approccio che si è affermato è invece guidato dai dati (*data-driven*) e si articola in tre passi: raccogliere un insieme di immagini corredate dalle rispettive etichette, utilizzare un algoritmo di apprendimento automatico per addestrare un classificatore su tali dati, e infine valutarne le prestazioni su immagini mai viste durante l'addestramento. Per questo motivo i dati disponibili vengono divisi in tre insiemi distinti: uno di addestramento, su cui il modello apprende i propri parametri, uno di validazione, utilizzato per confrontare configurazioni alternative e scegliere la migliore, e uno di test,

riservato alla valutazione finale.

I concetti richiamati in questa sezione vengono introdotti in letteratura nel contesto della classificazione, ma si estendono senza modifiche sostanziali ai compiti affrontati in questo lavoro, ovvero la localizzazione di un oggetto tramite bounding box e la stima di punti anatomici, descritti nella Sezione 1.4.

1.1.2 Dal classificatore lineare alle reti neurali

Il modello più semplice che realizza questo schema è il classificatore lineare, che calcola i punteggi delle classi come $f(x, W) = Wx+b$, dove x è l'immagine in ingresso rappresentata come vettore, W è una matrice di pesi e b un termine costante: a ciascuna classe corrisponde una riga di W, interpretabile come un modello di riferimento (*template*) per quella categoria. Disponendo di un solo template per classe, però, il classificatore lineare non riesce a rappresentare categorie che si presentano in forme diverse tra loro.

Una rete neurale supera questo limite introducendo un calcolo gerarchico. Nella sua forma più semplice, una rete a due strati calcola $f = W_2 \max(0, W_1 x)$: il primo strato produce una rappresentazione intermedia, detta strato nascosto (*hidden layer*), e il secondo la combina per ottenere i punteggi finali, apprendendo così più template intermedi condivisibili tra classi diverse anziché uno solo per categoria.

L'elemento che rende possibile questa maggiore espressività è la funzione applicata tra i due strati, detta funzione di attivazione. La sua presenza è essenziale: se i due strati fossero puramente lineari, la loro composizione $W_2 W_1 x$ equivarrebbe a una singola matrice $W_3 = W_2 W_1$, e il modello tornerebbe a essere un classificatore lineare. Tra le varie funzioni di attivazione proposte in letteratura, la ReLU [9], definita come $\max(0, x)$, è quella comunemente adottata come scelta predefinita.

Impilando più strati si ottiene una rete profonda, i cui strati completamente connessi (*fully connected*) collegano ogni unità a tutte quelle dello strato precedente. Il numero complessivo di unità determina la capacità del modello, ovvero la complessità delle funzioni che è in grado di rappresentare: una capacità elevata comporta il rischio di *overfitting*, che si contiene non riducendo la rete ma agendo sulla regolarizzazione, come descritto nella sezione seguente.

1.1.3 L'addestramento di una rete neurale

Definita la struttura del modello, resta da determinare il valore dei suoi parametri. Il problema si affronta in due passi: si definisce una funzione di

costo (*loss*) che quantifica quanto i punteggi prodotti dal modello si discostano da quelli attesi sui dati di addestramento, e si cerca poi un metodo efficiente per trovare i parametri che la minimizzano.

La loss complessiva è la media dei costi calcolati sui singoli esempi, alla quale si aggiunge un termine di regolarizzazione $\lambda R(W)$ che penalizza i pesi di valore elevato, dove λ ne controlla l'intensità; quando $R(W)$ è la somma dei quadrati dei pesi, questo termine è comunemente indicato come *weight decay*. Il primo termine spinge il modello ad aderire ai dati di addestramento, il secondo a non aderirvi eccessivamente, contenendo l'overfitting.

La minimizzazione avviene per discesa del gradiente: si calcola la derivata della loss rispetto a ciascun parametro e si aggiornano i parametri nella direzione opposta al gradiente, ripetendo il procedimento fino a convergenza. In una rete profonda questi gradienti vengono calcolati in modo efficiente tramite l'algoritmo di *backpropagation*, che propaga l'errore dall'uscita verso gli strati precedenti applicando la regola di derivazione delle funzioni composte [12].

Poiché calcolare il gradiente sull'intero dataset a ogni passo sarebbe troppo oneroso, in pratica lo si stima su piccoli sottoinsiemi di campioni estratti casualmente, detti *mini-batch*, la cui dimensione (*batch size*) è comunemente di 32, 64 o 128 elementi. Questa variante prende il nome di discesa del gradiente stocastica (SGD), e ciascun aggiornamento dei parametri calcolato su un mini-batch è detto *iterazione*: la loss può quindi essere valutata e ridotta più volte all'interno della stessa epoca, non solo al termine di un passaggio completo sui dati. L'entità di ciascun aggiornamento è regolata da un parametro detto *learning rate*. Un passaggio completo su tutti i dati di addestramento costituisce un'*epoca*, e l'addestramento si protrae per un numero prefissato di epoche oppure di iterazioni.

La SGD nella sua forma base può produrre aggiornamenti rumorosi e una convergenza meno regolare, a causa della stima del gradiente su un singolo mini-batch; per attenuare questi effetti si introduce il *momentum* [13], che accumula una media mobile dei gradienti dei passi precedenti per determinare la direzione di avanzamento, con un coefficiente tipicamente pari a 0.9.

1.1.4 Reti neurali convoluzionali

I layer completamente connessi visti finora ricevono in input l'immagine "appiattita" in un vettore, perdendo così la struttura spaziale a due dimensioni dei pixel. I layer convoluzionali affrontano il problema in modo diverso: l'input resta un volume tridimensionale, e un filtro di dimensioni ridotte viene fatto scorrere su tutte le posizioni spaziali, calcolando a ogni posizione un prodotto scalare tra il filtro e la porzione di immagine

sottostante. Questo permette alla rete di individuare pattern locali (bordi, texture) indipendentemente dalla loro posizione nell'immagine, mantenendo l'informazione spaziale che un layer fully-connected perderebbe. Questo meccanismo è specifico delle reti convoluzionali e non vale, ad esempio, per i Vision Transformer.

1.1.5 Transfer learning

Quando i dati di addestramento disponibili per un task specifico sono limitati, è comune riutilizzare un modello già addestrato su un dataset di grandi dimensioni (tipicamente ImageNet) invece di inizializzare i pesi casualmente, una pratica nota come *transfer learning*. Sono possibili due strategie: la *feature extraction*, in cui i pesi del modello pre-addestrato restano fissi e viene addestrato solo un nuovo layer finale specifico per il task; e il *fine-tuning*, in cui l'intera rete, pesi pre-addestrati compresi, continua a essere aggiornata durante l'addestramento sul nuovo task.

1.2 Approcci automatici alla stima dell'età ossea

Prima dell'avvento del deep learning, uno strumento automatico diffuso in ambito clinico è BoneXpert, un software commerciale sviluppato nel 2009 da Thodberg e Kreiborg [15], che stima l'età ossea tramite tecniche di elaborazione d'immagine classiche, precedenti al deep learning, basandosi sui criteri di G&P [3] e TW2 [14]. Essendo stato sviluppato su una coorte di pazienti danesi, la sua affidabilità non è garantita su popolazioni con caratteristiche diverse [1], una limitazione che motiva, tra l'altro, la costruzione di dataset come RHPE su popolazioni diverse da quelle già coperte.

Il dataset reso disponibile dalla RSNA Pediatric Bone Age Challenge del 2017 [4] ha successivamente favorito la diffusione di approcci basati su reti neurali convoluzionali, addestrate end-to-end direttamente sull'immagine radiografica per predire l'età ossea. Il paradigma BoNet, descritto nella sezione seguente, si inserisce in questo filone ma ne rappresenta un'evoluzione mirata, introducendo l'uso esplicito di informazione anatomica locale.

1.3 Il paradigma BoNet

Il paradigma proposto da BoNet [1] nasce da un'osservazione: i metodi clinici più precisi per la stima dell'età ossea, come il TW2, si basano su un'analisi locale di specifiche regioni anatomiche della mano. Sebbene alcuni approcci automatici esistenti al momento della pubblicazione tentassero già di sfruttare informazione locale, i dataset di bone age assessment disponibili

fornivano annotazioni solo a livello di immagine intera, senza indicare le regioni anatomiche rilevanti, rendendo difficile un'analisi locale precisa e sistematica come quella dei metodi clinici manuali. Gli autori propongono quindi di introdurre la stima di keypoint anatomici come nuovo task, per fornire al sistema un'informazione locale precisamente localizzata, in modo analogo a quella usata dai radiologi.

La pipeline è organizzata in tre stadi: prima la mano viene localizzata nell'immagine (hand detection), poi vengono stimati 17 keypoint, landmark anatomici posti in corrispondenza delle giunzioni tra due ossa (hand pose estimation), infine l'informazione ottenuta viene usata per stimare l'età ossea (bone age assessment). In quest'ultimo stadio, l'immagine ritagliata attorno alla mano e una mappa costruita a partire dai keypoint vengono combinate insieme all'informazione sul genere del paziente per produrre la stima finale (Figura 1.1).

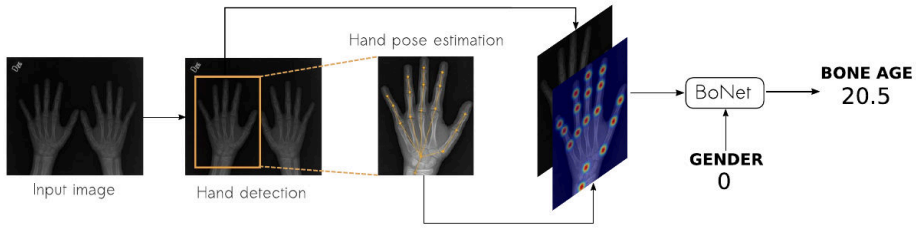


Figura 1.1: Schema della pipeline BoNet: l'immagine di partenza attraversa hand detection e hand pose estimation per individuare le regioni anatomiche di interesse; il crop della mano, la mappa costruita a partire dai keypoint e il genere del paziente costituiscono poi l'input dello stadio di bone age assessment. Figura tratta da Escobar et al. [1].

Gli autori mostrano che questo approccio, basato su informazione locale, supera nettamente le prestazioni di un modello che analizza solo l'immagine intera.

1.4 Architetture di riferimento

1.4.1 Hand detection: Faster R-CNN e ResNet

La prima fase della pipeline BoNet si basa su Faster R-CNN [10], un'architettura di object detection a due stadi. Una *Region Proposal Network* (RPN) scorre sulle feature map dell'immagine e propone un insieme di regioni candidate che potrebbero contenere un oggetto, senza ancora classificarle; una seconda rete prende in input queste proposte, classifica ciascuna regione e

ne raffina la bounding box. In questo lavoro, coerentemente con il paper di riferimento, Faster R-CNN viene utilizzato con backbone ResNet-50 [5] dotato di *Feature Pyramid Network* (FPN) [8], un'estensione che combina le feature a diverse risoluzioni per riconoscere meglio oggetti di dimensioni molto diverse, con pesi pre-addestrati sul dataset COCO.

Impilare più layer convoluzionali non migliora le prestazioni in modo automatico: oltre una certa profondità, una rete "semplice" (senza accorgimenti particolari) ottiene un errore peggiore sia in training sia in test rispetto a una rete meno profonda, segno che il problema non è overfitting ma difficoltà di ottimizzazione. ResNet risolve questo problema introducendo connessioni residue: anziché far apprendere a un blocco di layer la trasformazione desiderata $H(x)$ direttamente, lo si fa apprendere il residuo $F(x) = H(x) - x$, sommato poi a x tramite una connessione diretta (*shortcut*). In questo modo, se i layer aggiuntivi non sono utili, è sufficiente che imparino $F(x) = 0$ per ottenere la funzione identità, garantendo che una rete più profonda non sia mai peggiore di una più superficiale. Questo rende ResNet una scelta comune come backbone per compiti di visione artificiale.

1.4.2 Keypoint estimation: Simple Baselines e Keypoint R-CNN

Per il secondo stadio della pipeline, gli autori di BoNet si basano su Simple Baselines for Human Pose Estimation [16], descritta nel paper di riferimento come l'architettura allo stato dell'arte per la stima della posa umana al momento della sua pubblicazione.

Un'alternativa è Keypoint R-CNN [6], presentata nello stesso paper di Mask R-CNN come estensione dello stesso framework alla stima della posa: ogni keypoint viene predetto individuando la sua posizione più probabile all'interno di una griglia di possibili posizioni (una *maschera* binaria, la stessa rappresentazione usata in Mask R-CNN per la segmentazione). A differenza di Simple Baselines, che richiede in input un crop già ritagliato attorno al soggetto, Keypoint R-CNN integra detection e keypoint estimation in un'unica rete, individuando prima le regioni candidate con la RPN e poi predicendo i keypoint direttamente su ciascuna di esse. Per farlo, ogni regione candidata, che può avere dimensioni diverse a seconda di dove si trova la mano nell'immagine, deve prima essere convertita in una griglia di dimensione fissa su cui operare. L'operazione impiegata a questo scopo nelle architetture precedenti, *ROIPool*, arrotondava i confini della regione alle celle più vicine della griglia, introducendo così un piccolo errore di posizione. Mask R-CNN, e con esso Keypoint R-CNN, la sostituisce con una tecnica più precisa, *ROIAlign*, che evita questo arrotondamento e stima i valori nelle posizioni esatte: gli autori di Mask R-CNN mostrano che questa

maggiore precisione conta soprattutto per i keypoint, che richiedono una localizzazione più fine rispetto alla semplice classificazione.

Le due architetture effettivamente impiegate in questo lavoro, Faster R-CNN per la detection e Keypoint R-CNN per la keypoint estimation (Simple Baselines non viene reimplementata), vengono utilizzate tramite *fine-tuning* (Sez. 1.1.5): i pesi pre-addestrati sul dataset COCO vengono mantenuti, ma l'ultimo layer specifico per il task viene sostituito con uno nuovo, dimensionato per il numero di classi o di keypoint del problema, e l'intera rete viene poi addestrata end-to-end sul dataset RHPE. Il fine-tuning è generalmente preferibile alla feature extraction quando il dominio delle immagini di destinazione si discosta significativamente da quello di pre-addestramento, permettendo alla rete di adattare anche le feature di più basso livello: le radiografie del dataset RHPE, in scala di grigi e con caratteristiche visive molto diverse dalle fotografie naturali di COCO, rientrano in questo caso.

1.4.3 Evaluation metrics

IoU. Per decidere se una bounding box predetta sia sufficientemente vicina a quella reale serve una misura di similarità e una soglia oltre la quale considerare la predizione corretta. Per i box la misura utilizzata è l'Intersection over Union (IoU): il rapporto tra l'area di sovrapposizione tra predizione e ground truth e l'area della loro unione (Figura 1.2), pari a 1 quando le due box coincidono e a 0 quando non si sovrappongono affatto. Una soglia IoU di 0.5 richiede che le due box si sovrappongano per almeno metà della loro area unita per essere contate come corrette; una soglia di 0.95 richiede una sovrapposizione quasi perfetta.

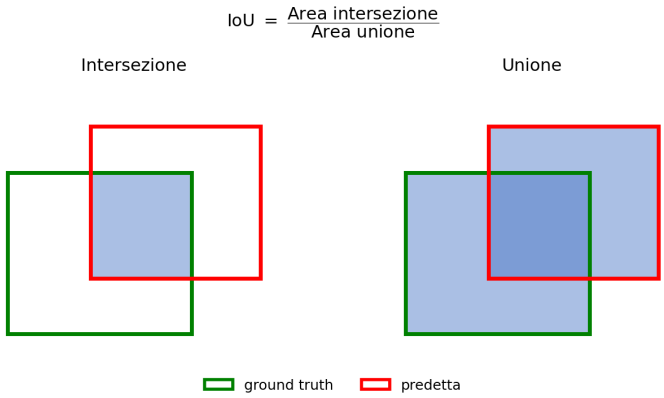


Figura 1.2: Le due aree usate nel calcolo dell'IoU tra una bounding box predetta e quella ground truth.

Precisione, richiamo e AP. Fissata una soglia, si ordinano le predizioni del modello per punteggio di confidenza e si varia via via quante accettarne: al crescere della soglia di confidenza cambiano sia la precisione (quale frazione delle predizioni accettate è corretta) sia il richiamo (quale frazione degli oggetti reali viene individuata), tracciando una curva precisione-richiamo. L'area sottesa a questa curva è l'*Average Precision* (AP) per una classe.

mAP e mAR. Mediando le AP su tutte le classi si ottiene la mean Average Precision (mAP) e, allo stesso modo, la mean Average Recall (mAR). Nei task di questo lavoro esiste una sola classe di primo piano (la mano), per cui la mAP coincide numericamente con l'AP.

mAP@.50 e mAP@[.5:.95]. La notazione mAP@.50 indica l'AP calcolata con un'unica soglia IoU pari a 0.5; mAP@[.5:.95] indica invece che l'AP viene calcolata su dieci soglie IoU (da 0.5 a 0.95, con passo 0.05) e mediata, risultando quindi più severa: penalizza anche le predizioni imprecise, non solo quelle completamente sbagliate.

OKS. Per la keypoint estimation la misura di similarità analoga è l'Object Keypoint Similarity (OKS), introdotta con il benchmark di keypoint detection di COCO [7] e analizzata in dettaglio in [11]: anziché un'area di sovrapposizione, misura la distanza tra il keypoint predetto e quello reale, normalizzata rispetto alla scala dell'oggetto e a un parametro σ specifico per ciascun keypoint (Sez. 2.4.1): a parità di distanza in pixel, la stessa imprecisione viene quindi penalizzata in modo diverso a seconda del keypoint (Figura 1.3).

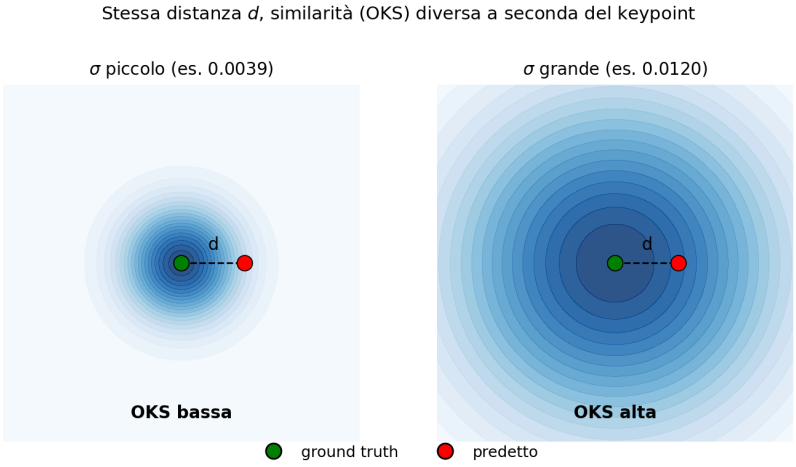


Figura 1.3: A parità di distanza d tra keypoint predetto e ground truth, un σ più piccolo (a sinistra) penalizza l'errore più duramente di un σ più grande (a destra). I valori di σ mostrati sono i valori minimo e massimo tra quelli usati in questo lavoro (Tabella 2.2).

mAP su OKS. La notazione mAP@[.5:.95] si applica allo stesso modo, sostituendo semplicemente le soglie di IoU con soglie di OKS: è quindi la stessa ricetta di calcolo applicata a una misura di similarità diversa, non una metrica diversa. Nei risultati riportati nel Capitolo 3, la mAP calcolata sui box è sempre basata su IoU e quella calcolata sui keypoint su OKS.

1.5 Dataset di riferimento

Il dataset RHPE (Radiological Hand Pose Estimation) [1], utilizzato in questo lavoro, è composto da radiografie della mano di pazienti pediatrici della Fundación Santa Fe de Bogotá (Colombia), annotate con bounding box della mano sinistra (non dominante) e posizione dei 17 keypoint anatomici utilizzati dal paradigma BoNet, oltre all'età ossea (stabilita da due radiologi esperti per ciascun paziente), età cronologica e genere.

Capitolo 2

Materiali e metodi

2.1 Dataset RHPE

Il dataset RHPE (Radiological Hand Pose Estimation) è composto da radiografie della mano di pazienti pediatrici, suddivise nei tre split standard di training, validation e test (Sez. 1.1.1). Le annotazioni sono fornite in due formati:

- **JSON (formato COCO):** bounding box della mano e 17 keypoint anatomici codificati come triplette (x, y, v), dove v rappresenta il flag di visibilità;
- **CSV:** età ossea (*bone age*), età cronologica e genere per ogni paziente.

Il test set non include l'età ossea, poiché è il valore che il sistema deve predire.

Le radiografie della mano acquisite in ambito clinico mostrano tipicamente entrambe le mani sulla stessa immagine, per permettere un confronto diretto tra i due lati, ed è così anche per le immagini fornite da RHPE: ogni file mostra entrambe le mani, ma le annotazioni (bounding box e keypoint) riguardano esclusivamente la mano sinistra (non dominante), presente per convenzione nella metà sinistra dell'immagine. Questo aspetto è rilevante per il lavoro di estensione alla mano destra descritto in Sez. 2.3.5.

2.1.1 Analisi esplorativa

La suddivisione nei tre split è fortemente asimmetrica (Tabella 2.1): 5491 immagini di training, 713 di validation e sole 79 di test. La composizione per genere è invece simile tra training e validation, con una prevalenza femminile attorno al 57%, mentre il test set risulta più bilanciato (51.9%).

La Figura 2.1 riporta la distribuzione dell'età ossea e dell'età cronologica nei tre split. Entrambe si concentrano attorno ai 120–140 mesi e si estendono su quasi tutto l'arco di età considerato dal task, con forme confrontabili tra training e validation; il test set, contando poche decine di immagini, mostra una distribuzione più irregolare per effetto della dimensione ridotta. Il pannello relativo all'età ossea del test set è vuoto perché questa annotazione non è disponibile per tale split, essendo il valore che il sistema deve predire.

Tabella 2.1: Distribuzione del dataset RHPE per split.

Split	Immagini	Maschi	Femmine
Train	5491	2372 (43.2%)	3119 (56.8%)
Val	713	306 (42.9%)	407 (57.1%)
Test	79	38 (48.1%)	41 (51.9%)

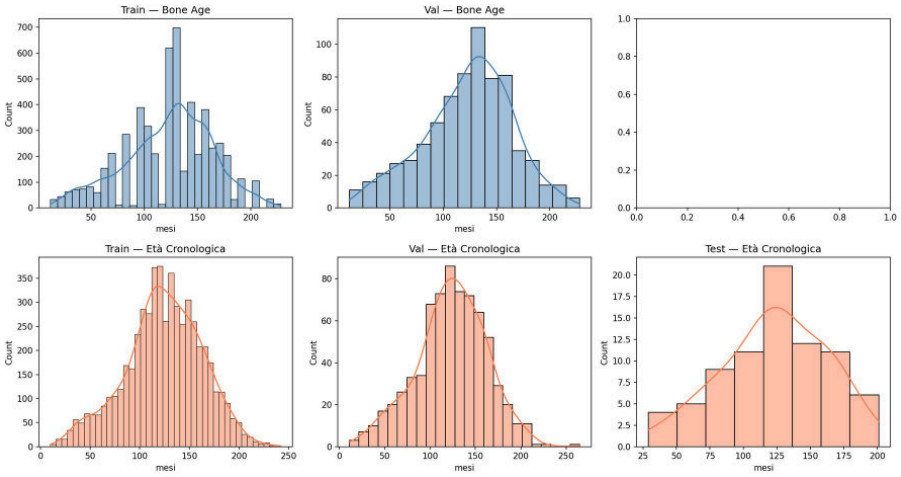


Figura 2.1: Distribuzione di bone age ed età cronologica sui tre split del dataset RHPE.

2.2 Hand Detection

Il primo stadio della pipeline consiste nel localizzare automaticamente la mano nell'immagine tramite bounding box. Seguendo il paper BoNet, il modello scelto è Faster R-CNN con backbone ResNet-50 FPN, con pesi pre-addestrati sul dataset COCO (Sez. 1.4). L'architettura viene adattata al task tramite fine-tuning: l'ultimo layer (`box_predictor`), originariamente configurato per 91 classi (dataset COCO), viene sostituito con uno nuovo per 2 classi (background e mano). L'intero flusso descritto in questa sezione,

dal campionamento dei dati al ciclo di addestramento, è implementato nello script `train_bbox.py`.

2.2.1 Campionamento stratificato

Su indicazione del supervisore, i sottoinsiemi utilizzati per la hand detection sono composti da 250 immagini di training e 50 di validation. Anziché utilizzare le prime N immagini del training set, per la costruzione dei sottoinsiemi di training e validation è stato adottato un campionamento stratificato per fascia d'età, con fasce di ampiezza 24 mesi. Questa scelta garantisce che il sottoinsieme utilizzato per l'addestramento sia rappresentativo dell'intera distribuzione delle età del dataset, evitando che alcune fasce risultino sovra- o sotto-rappresentate rispetto ad altre.

2.2.2 Collegamento tra annotazioni CSV e JSON

Un aspetto non banale nella costruzione del dataset riguarda il collegamento tra le informazioni provenienti dai due formati di annotazione disponibili. Il file CSV identifica ciascun paziente tramite un ID progressivo (1, 2, 3, …), mentre il file JSON in formato COCO assegna a ciascuna immagine un ID interno arbitrario (ad esempio 3722) e un nome file (ad esempio `00001.png`). L'unico elemento in comune tra le due annotazioni è il numero progressivo codificato nel nome del file, che corrisponde all'ID utilizzato nel CSV. Il dataset è stato quindi costruito estraendo tale numero dal nome del file e utilizzandolo per recuperare, dal CSV, le informazioni associate alla stessa immagine. Si tratta di un dettaglio implementativo apparentemente marginale, ma che, se gestito in modo scorretto, introduce silenziosamente disallineamenti tra le immagini e le relative annotazioni.

2.2.3 Iperparametri

Gli iperparametri di training sono stati allineati a quelli riportati nel paper BoNet: ottimizzatore SGD con learning rate costante pari a 2.5×10^{-3}, momentum 0.9 e weight decay 5×10^{-4}, per un totale di 5000 iterazioni con batch size 4 (Sez. 1.4.3 per la definizione di mAP@[.5:.95], usata per selezionare il modello migliore come descritto di seguito).

2.2.4 Ciclo di addestramento

Ad ogni iterazione, il modello riceve in input un mini-batch di immagini e i relativi target, ciascuno costituito dalla bounding box (convertita dal formato COCO $[x, y, w, h]$ al formato $[x_1, y_1, x_2, y_2]$ atteso da torchvision) e

da un'etichetta di classe, sempre pari a 1 poiché è definita un'unica classe di primo piano (*mano*). Poiché il numero di oggetti annotati per immagine può variare, le immagini di un batch non possono essere impilate direttamente in un unico tensore come fa di default il `DataLoader`: viene quindi fornita una funzione di collazione dedicata, che si limita a raccogliere immagini e target in due liste separate anziché impilarli.

Il comportamento dei modelli di detection di torchvision dipende dalla modalità in cui si trovano: in modalità di addestramento (`model.train()`), avendo a disposizione i target, il modello non restituisce predizioni ma un dizionario con quattro componenti di loss, relative a classificazione, regressione della bounding box, objectness e raffinamento delle proposte della RPN: classificazione e objectness sono funzioni di tipo *cross-entropy*: se il modello assegna una probabilità bassa alla classe corretta la penalità è alta, se la assegna vicina a 1 la penalità è quasi nulla. Regressione della bounding box e raffinamento delle proposte sono invece di tipo *smooth L1*, che misura la distanza tra le coordinate predette e quelle reali: per piccoli errori si comporta come l'errore quadratico, ma per errori grandi cresce più lentamente, così un singolo caso molto sbagliato non domina l'intera loss. La somma delle quattro costituisce la loss totale su cui viene calcolato il gradiente; in modalità di valutazione (`model.eval()`) restituisce invece le predizioni.

Le 5000 iterazioni indicate dal paper BoNet si riferiscono a singoli passi di aggiornamento dei parametri, non a epoche complete. Con un sottoinsieme di sole 250 immagini di training e batch size 4, un'epoca corrisponde a $\lceil 250/4 \rceil = 63$ iterazioni (l'ultimo batch dell'epoca contiene solo 2 immagini); raggiungere le 5000 iterazioni richiede quindi di ripercorrere lo stesso sottoinsieme circa 80 volte. Al termine di ogni epoca il modello viene valutato sul validation set, e i pesi vengono salvati solo se la mAP@[.5:.95] migliora rispetto al massimo osservato fino a quel momento.

Gli esperimenti sono stati eseguiti sul server escher, dotato di due GPU NVIDIA GeForce RTX 3060 da 12 GB di memoria ciascuna; ogni addestramento utilizza una sola GPU.

2.3 Keypoint Estimation

Il secondo stadio della pipeline consiste nello stimare la posizione dei 17 keypoint anatomici della mano. Il paper BoNet utilizza Simple Baselines for Human Pose Estimation [16] con backbone ResNet-50, un'architettura encoder-decoder che riceve in input un crop della mano e produce 17 heatmap tramite layer deconvoluzionali.

Per questo lavoro si è scelto di utilizzare Keypoint R-CNN [6] (Sez. 1.4), già disponibile in torchvision, come alternativa pragmatica a Simple Base-

lines. La scelta, concordata con il supervisore, è motivata da: interfaccia analoga a Faster R-CNN (già utilizzato per la detection) e disponibilità immediata senza necessità di implementare l'architettura da zero. Come per la hand detection, l'intero addestramento, incluse le quattro varianti descritte di seguito, è implementato in un unico script, `train_keypoints.py`, parametrizzato tramite il flag `--mode`.

2.3.1 Quattro varianti di modello

Durante lo sviluppo sono state esplorate quattro varianti del modello, ciascuna con una diversa strategia di training:

`full` addestrato sull'immagine intera, con bounding box e keypoint nelle coordinate originali. Poiché Keypoint R-CNN integra detection e keypoint estimation, questo modello è in grado di localizzare la mano autonomamente, senza bisogno di un crop esterno.

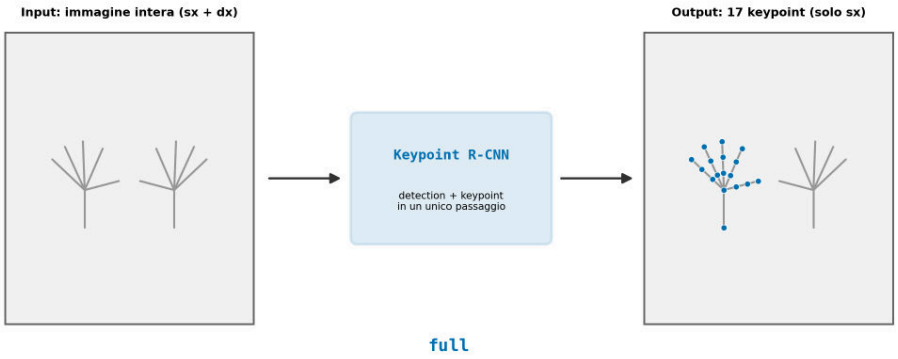


Figura 2.2: Processo di training/inferenza del modello `full`: l'immagine intera contiene entrambe le mani, ma solo la sinistra è annotata in RHPE e viene quindi predetta.

`gt_crop` addestrato su crop ottenuti tramite la bounding box ground truth, replicando la metodologia del paper BoNet (che valuta esplicitamente le prestazioni con crop *ground truth* come upper bound del task). Il target di bounding box coincide sempre con l'intero crop (`[0, 0, w, h]`).

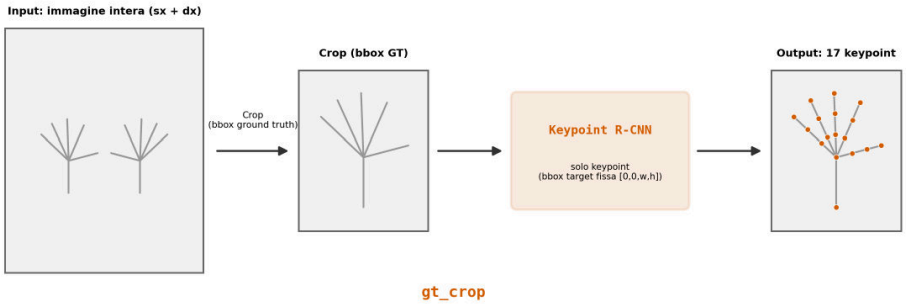


Figura 2.3: Processo di training/inferenza del modello `gt_crop`: il crop, ottenuto dalla bounding box ground truth e non da un modello di detection, isola la sola mano sinistra prima della keypoint estimation.

`mirrored` estensione di `gt_crop` pensata per predire keypoint anche sulla mano destra, assente nelle annotazioni RHPE (Sez. 2.1). Per ogni immagine di training, oltre al crop originale viene aggiunta una versione specchiata orizzontalmente (tramite `ImageOps.mirror`), con le coordinate x dei keypoint ribaltate come $x' = w - x$ (dove w è la larghezza del crop). Il dataset di training raddoppia così da 250 a 500 campioni.

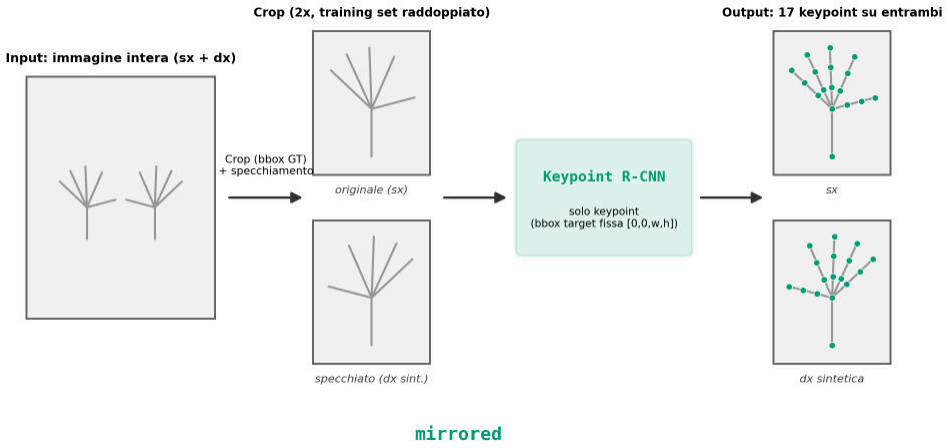


Figura 2.4: Processo di training/inferenza del modello `mirrored`: al crop ground truth si affianca una sua versione specchiata, che simula una mano destra sintetica; lo stesso modello produce keypoint su entrambi.

`mirrored_full` stessa idea di `mirrored`, ma applicata all'immagine intera anziché al crop, e quindi in grado di eseguire detection e keypoint estimation in un unico passaggio come il modello `full`: bounding box e

keypoint vengono specchiati usando la larghezza dell'intera immagine ($x' = W - x$) invece della larghezza del crop. Questa variante è stata introdotta per risolvere un limite di `mirrored`: quel modello, avendo visto in training solo crop già stretti attorno alla mano (con bounding box target sempre coincidente con l'intero input), non aveva mai imparato a localizzare con precisione una mano all'interno di un contesto più ampio con sfondo (Sez. 2.3.5).

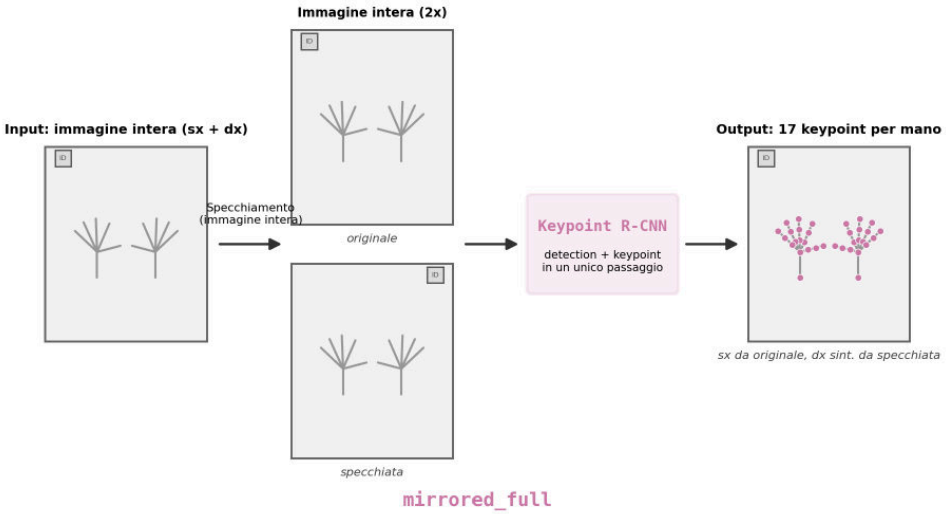


Figura 2.5: Processo di training/inferenza del modello `mirrored_full`: a specchiarsi è l'intera immagine, non solo il crop. Poiché le due mani sono disposte specularmente l'una rispetto all'altra, lo specchiamento dell'immagine lascia invariato l'aspetto di ciascuna mano (a cambiare è solo quale mano fisica occupa quale posizione), come reso visibile dalla posizione dell'adesivo con l'ID paziente, che si sposta da sinistra a destra. Lo stesso modello `full` applicato a entrambe le versioni produce keypoint per la mano sinistra reale e per quella destra sintetica.

2.3.2 Campionamento e iperparametri

Il campionamento segue la stessa logica stratificata per fascia d'età utilizzata per la hand detection (Sez. 2.2): 250 immagini di training e 50 di validation. Lo script supporta anche l'addestramento sull'intero training set tramite un'opzione dedicata (`--full-dataset`), utilizzata per i retraining discussi in Sez. 3.2.1 e 3.2.2.

Il paper BoNet indica, per la keypoint estimation, solo il numero di epoche (20) e l'uso dei "parametri suggeriti" da Simple Baselines (Adam,

learning rate 10^{-3}). Poiché Keypoint R-CNN appartiene alla famiglia Faster R-CNN, tradizionalmente addestrata con SGD, questi parametri non si applicano direttamente all'architettura scelta. Si è quindi optato per SGD con iperparametri standard per questa famiglia di modelli (learning rate 5×10^{-3}, momentum 0.9, weight decay 5×10^{-4}), mantenendo le 20 epoche indicate dal paper.

Durante l'analisi del confronto con BoNet (Capitolo 4) è emerso che il paper di Simple Baselines specifica, oltre ai parametri di ottimizzazione già discussi, anche uno schema di data augmentation applicato in training: rotazione casuale ($\pm 40°$), scala casuale ($\pm 30\%$) e flip orizzontale. Il codice di questo lavoro non applicava alcuna augmentation stocastica per immagine; il flip presente nelle varianti `mirrored`/`mirrored_full` è deterministico e a livello di dataset (una copia specchiata aggiunta per ogni campione, Sez. 2.3.4), pensato per generare dati di training per la mano destra, non come tecnica di regolarizzazione per la sinistra. Si è quindi aggiunta la stessa rotazione/scala casuale usata da Simple Baselines, applicata all'immagine intera e alle relative coordinate di bounding box e keypoint prima di qualsiasi elaborazione specifica della modalità (crop, mirroring), e solo al training set, non alla validazione. Il flip non è stato incluso: applicarlo a `gt_crop` richiederebbe che il modello imparasse a riconoscere anche l'orientamento speculare della mano, snaturando la variante e avvicinandola concettualmente a `mirrored`. Questa augmentation è stata introdotta in una fase successiva del lavoro: i risultati che ne fanno uso sono indicati esplicitamente nel Capitolo 3.

2.3.3 Ciclo di addestramento

Rispetto alla hand detection, il target di ciascuna immagine include anche i keypoint, rappresentati come un tensore di forma $(1, 17, 3)$: per ciascuno dei 17 keypoint anatomici, le prime due componenti sono le coordinate (x, y) e la terza è un flag di visibilità. In modalità di addestramento, il modello restituisce quindi un dizionario con cinque componenti di loss anziché quattro: alle stesse quattro della hand detection (Sez. 2.2) si aggiunge una loss specifica per i keypoint (anch'essa una *cross-entropy*, calcolata non sulle coordinate continue ma sulla posizione discretizzata di ciascun keypoint in un'heatmap), e la somma delle cinque costituisce la loss totale su cui viene calcolato il gradiente.

Il criterio di selezione del checkpoint migliore segue inizialmente la stessa logica della hand detection (Sez. 2.2): si salvano i pesi del modello quando la mAP@[.5:.95] sul validation set migliora rispetto al massimo osservato fino a quel momento. Per le varianti `gt_crop` e `mirrored` questo criterio si è però rivelato poco informativo: il target di bounding box coincide sempre con l'intero crop (`[0, 0, w, h]`, Sez. 2.3.1), quindi la componente di detection

della mAP satura rapidamente e smette di distinguere tra epoche successive, mentre la qualità della stima dei keypoint, la parte del task che qui interessa davvero, continua a variare in modo non catturato dalla metrica. Si è quindi adottato un criterio basato sulla loss di validazione complessiva, la somma delle cinque componenti descritte sopra (calcolata senza aggiornare i pesi, tramite un passaggio in avanti in modalità di addestramento), che include esplicitamente il contributo dei keypoint ed è una pratica comune per la selezione del modello durante il training [2]. Il nuovo criterio è stato adottato a partire dal retraining di `gt_crop` sul dataset completo (Sez. 3.2.1); i risultati sul sottoinsieme da 250 immagini riportati in Tabella 3.2, per tutte e quattro le varianti, restano invece selezionati con il criterio precedente basato sulla mAP.

2.3.4 Estensione alla mano destra tramite mirroring

Poiché le annotazioni RHPE coprono esclusivamente la mano sinistra (Sez. 2.1), l'unico modo per ottenere dati di training con ground truth per la mano destra è generarli sinteticamente tramite specchiamento orizzontale, sia dell'immagine che delle coordinate dei keypoint. Questa strategia, concordata con il supervisore, è stata implementata a due livelli, sul crop (`mirrored`) e sull'immagine intera (`mirrored_full`), con risultati qualitativamente diversi discussi nel Capitolo 3.

La valutazione quantitativa sulla mano destra non è possibile, in assenza di ground truth nel test set RHPE; la verifica è quindi qualitativa, tramite ispezione visiva delle predizioni.

2.3.5 Esplorazione su entrambe le mani: metodologia

Al di là del confronto quantitativo tra le quattro varianti di modello, è stata condotta un'esplorazione qualitativa mirata a ottenere keypoint corretti su entrambe le mani nella stessa immagine, attraverso quattro approcci successivi, ciascuno motivato dai limiti del precedente e implementato come una modalità (`--version`) dello stesso script esplorativo, `right_hand_keypoints.py`. Il punteggio di confidenza associato a ciascuna detection, citato più volte nel seguito, è l'output diretto del classificatore del modello (la probabilità stimata che una regione contenga una mano), calcolato allo stesso modo indipendentemente dalla disponibilità di un ground truth. In questa esplorazione qualitativa le predizioni non vengono infatti confrontate con alcuna annotazione di riferimento, né per la mano sinistra né per la destra: il confronto quantitativo con il ground truth disponibile per la mano sinistra è invece descritto in Sez. 2.4, ed è quello alla base dei risultati riportati nella Tabella 3.2.

v1 Come primo tentativo, la mano destra viene cercata affidandosi al modello `full` (Sez. 2.3.1), che integrando detection e keypoint estimation può in linea di principio produrre più di una detection per immagine: si prendono le prime due predizioni per punteggio di confidenza, e per ciascun crop così ottenuto si stimano i keypoint con il modello `mirrored`.

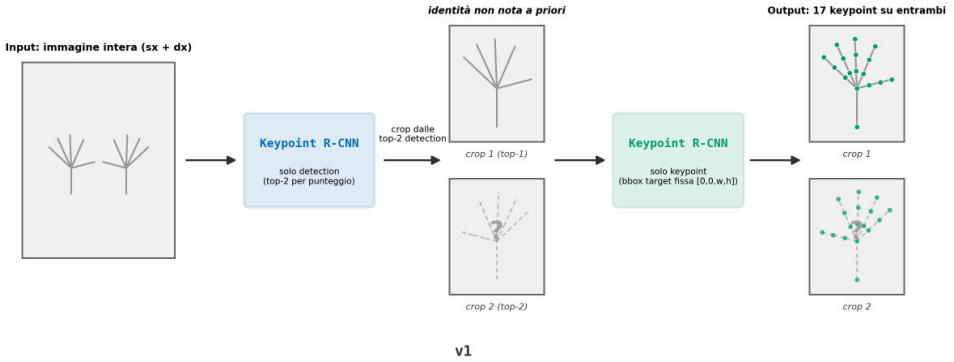


Figura 2.6: Processo di v1: il modello `full`, usato solo per la detection, produce le due predizioni con punteggio più alto; su ciascun crop così ottenuto si stimano i keypoint con `mirrored`. Il secondo crop è rappresentato con una forma generica anziché con una mano riconoscibile, perché nulla nella pipeline assegna un lato alle due detection: può trattarsi della mano destra, di un doppione della sinistra, o di nessuna delle due (Sez. 3.3).

v2 Poiché `full` non è mai stato addestrato a trovare più di una mano per immagine, si è provato a semplificare il problema dividendo verticalmente l'immagine a metà (sfruttando la convenzione RHPE per cui l'immagine intera corrisponde alla metà sinistra della radiografia) ed eseguendo la detection con `full` separatamente su ciascuna metà, così che il modello debba individuare al più una mano per volta, più vicino a ciò che ha visto in training. I keypoint vengono stimati, come in v1, con `mirrored`.

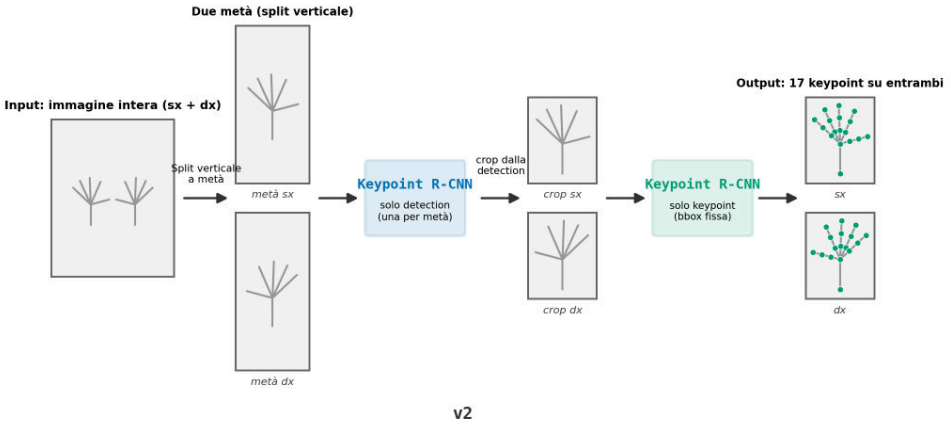


Figura 2.7: Processo di v2: l'immagine viene divisa verticalmente a metà prima della detection con `full`, in modo che il modello debba individuare al più una mano per metà; i keypoint sono stimati, come in v1, con `mirrored` sui crop risultanti.

v3 Poiché i risultati di v2 restano incostanti, si sostituisce `full` con `mirrored` anche per la detection sulle due metà, nell'ipotesi che, avendo visto in training sia l'orientamento originale sia quello specchiato, possa generalizzare meglio nel riconoscere una mano reale destra.

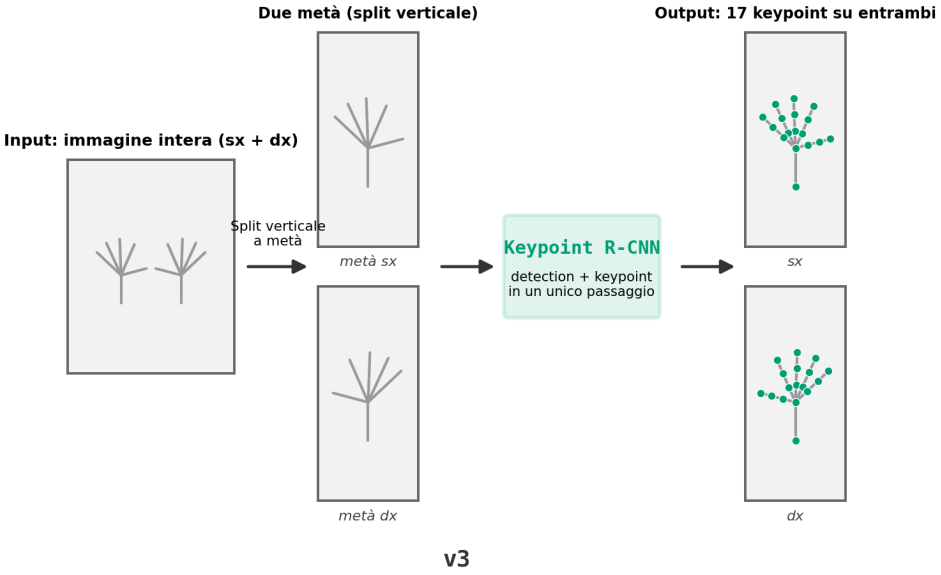


Figura 2.8: Processo di v3: rispetto a v2, `mirrored` sostituisce `full` anche per la detection, eseguendo detection e keypoint estimation in un unico passaggio su ciascuna metà; lo stadio di crop separato non è più necessario.

v4 Infine, si utilizza un singolo modello, `mirrored_full`, per detection e keypoint estimation in un unico passaggio sull'immagine intera, senza alcuna suddivisione né pipeline a più stadi.

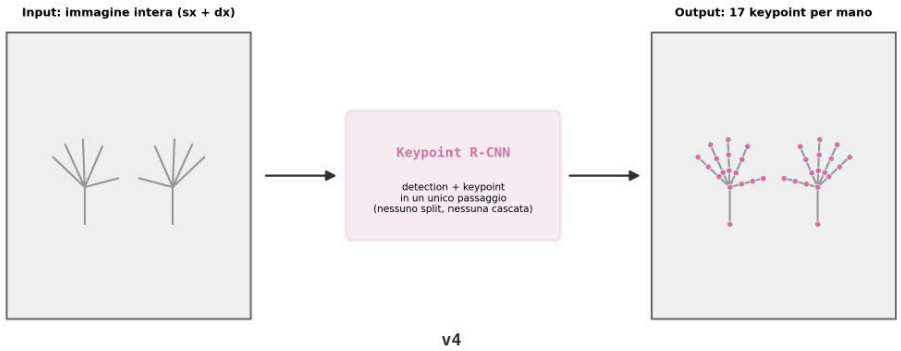


Figura 2.9: Processo di v4: nessuno split e nessuna cascata di modelli, un unico passaggio di `mirrored_full` sull'immagine intera produce direttamente i keypoint per entrambe le mani. Il meccanismo interno dello specchiamento su cui si basa `mirrored_full` è illustrato in dettaglio in Figura 2.5.

I risultati dei quattro approcci sono discussi in Sez. 3.3.

2.4 Metriche di valutazione

Le definizioni di IoU, mAP, mAR e OKS utilizzate in questo lavoro sono descritte in Sez. 1.4.3. Per la hand detection, mAP e mAR vengono calcolate con la libreria `torchmetrics`: lo script `eval_bbox.py` carica il modello addestrato per lo split scelto (`--split`), ne raccoglie le predizioni sull'intero split e le confronta con il ground truth; con il flag opzionale `--visualize` salva anche alcune immagini di esempio con la bounding box predetta (rossa) e quella reale (verde) sovrapposte, per un'ispezione qualitativa.

Per la keypoint estimation, la libreria `torchmetrics` (v1.5.2) non supporta `iou_type='keypoints'`; la valutazione basata su OKS è stata quindi implementata direttamente con `pycocotools.cocoeval.COCOeval`. Lo script `eval_keypoints.py` seleziona il modello (`--model`) e la modalità di input (`--mode`) da valutare, elabora ogni immagine dello split scelto di conseguenza (immagine intera, crop con bounding box ground truth, oppure crop con bounding box predetta dal modello di detection), confronta le predizioni con il ground truth tramite COCOeval, e registra i risultati in un file centralizzato (`all_metrics.json`) insieme a quelli delle altre combinazioni già calcolate; con il flag opzionale `--visualize` salva inoltre alcune immagini di esempio con i keypoint predetti (rossi) e quelli reali (verdi) sovrapposti.

Un aspetto rilevante per la riproducibilità riguarda la modalità `full`. Poiché Keypoint R-CNN può restituire più di una detection per immagine, mentre il ground truth annota la sola mano sinistra (Sez. 2.1), è necessario un criterio per scegliere quale predizione confrontare. Fra le detection con punteggio di confidenza superiore a 0.5 viene selezionata quella con la bounding box più a sinistra, coerentemente con la convenzione RHPE per cui la mano annotata occupa la metà sinistra dell'immagine; se nessuna detection supera la soglia, si ricade sulla predizione con il punteggio più alto. Il filtro sulla confidenza è necessario perché la sola posizione non basta: le radiografie contengono elementi spuri, come le etichette identificative, che il modello può occasionalmente rilevare con punteggio basso e che si trovano più a sinistra della mano. Nelle modalità `gt_crop` e `det_crop` il problema non si pone, poiché l'input è già ritagliato attorno a una singola mano; in `det_crop` il crop viene costruito a partire dalla detection con punteggio più alto prodotta dal modello di hand detection.

2.4.1 Sigma OKS

Il parametro σ regola la severità della metrica OKS: valori più piccoli penalizzano più duramente piccoli errori di posizionamento. I sigma standard

definiti per il benchmark COCO [7, 11] sono calibrati sui keypoint del corpo umano e non hanno un equivalente diretto per i keypoint anatomici della mano.

In una prima fase è stato utilizzato un sigma uniforme (0.05) come approssimazione documentata. Successivamente, dal materiale supplementare del paper BoNet è stato possibile recuperare i 17 valori di σ effettivamente utilizzati dagli autori, calcolati tramite la varianza delle annotazioni di più esperti sulle stesse immagini (Tabella 1 del materiale supplementare). Questi valori, riportati in Tabella 2.2, sono circa un ordine di grandezza più stretti di quelli standard COCO (range 0.0039–0.0120 contro 0.025–0.107), riflettendo la maggiore precisione richiesta nella localizzazione dei landmark anatomici della mano.

Tabella 2.2: Sigma OKS per i 17 keypoint della mano, dal materiale supplementare del paper BoNet [1].

Keypoint	σ
f1_0, f1_1, f1_2	0.0110, 0.0107, 0.0090
f2_0, f2_1, f2_2	0.0076, 0.0074, 0.0071
f3_0, f3_1, f3_2	0.0039, 0.0039, 0.0040
f4_0, f4_1, f4_2	0.0070, 0.0071, 0.0071
f5_0, f5_1, f5_2	0.0120, 0.0120, 0.0120
anchor_0, anchor_1	0.0090, 0.0080

L'adozione dei sigma reali ha comportato una revisione sostanziale dei risultati numerici (Sez. 3.2), portandoli a valori più bassi ma molto più confrontabili con quelli riportati dal paper originale.

Capitolo 3

Risultati sperimentali

3.1 Hand Detection

Tabella 3.1: Risultati del modello di hand detection sui tre split. La mAP è calcolata su soglie di IoU (Sez. 1.4.3).

Split	mAP@[.5:.95]	mAP@.50	mAP@.75
Train	0.9159	0.9888	0.9866
Val	0.8856	0.9842	0.9443
Test	0.9219	0.9791	0.9791

I risultati sono consistenti tra i tre split, senza segnali di overfitting. Il paper BoNet [1] riporta mAP@[.5:.95] = 90.1% sul validation set con l'intero training set; il valore ottenuto (88.56%) è comparabile nonostante l'addestramento su un sottoinsieme di sole 250 immagini. Per un task relativamente semplice come la localizzazione della mano, utilizzare l'intero training set non è efficiente: addestrare su un sottoinsieme ben campionato riduce in modo significativo tempi e costi di addestramento, a fronte di prestazioni comparabili e di una perdita minima. La Figura 3.1 mostra alcuni esempi di bounding box predetta e ground truth sul test set.

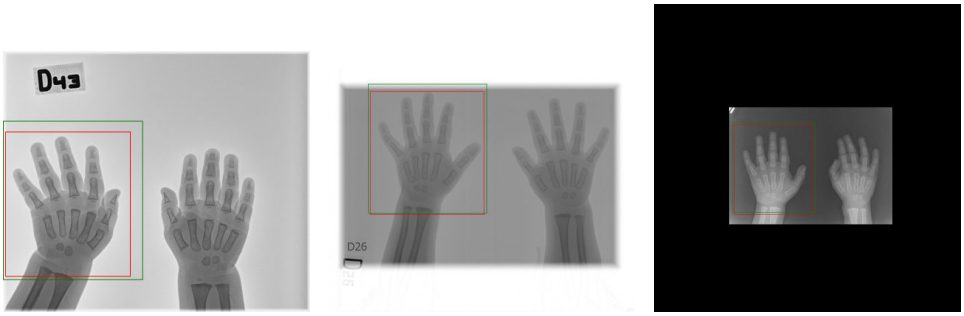


Figura 3.1: Esempi di bounding box predetta (rossa) e ground truth (verde) sul test set.

3.2 Keypoint Estimation

Le quattro varianti di modello descritte in Sez. 2.3.1 sono state valutate su tutti e tre gli split, in tre modalità di input: `full` (immagine intera), `gt_crop` (crop con bounding box ground truth) e `det_crop` (crop con bounding box predetta dal modello di detection, Sez. 2.2). I risultati riportati usano i sigma OKS reali del paper (Tabella 2.2).

Tabella 3.2: Risultati di keypoint estimation sul test set, con i sigma OKS del paper. La mAP è calcolata su soglie di OKS, non di IoU (Sez. 1.4.3).

Model	Mode	mAP@[.5:.95]	mAP@.50	mAP@.75	mAR@[.5:.95]
full	full	0.6986	0.9702	0.9061	0.7456
	gt_crop	0.5153	0.8113	0.5550	0.5608
	det_crop	0.5407	0.8964	0.5887	0.5937
gt_crop	full	0.0856	0.2446	0.0484	0.1013
	gt_crop	0.7037	0.9619	0.9239	0.7456
	det_crop	**0.7205**	**0.9713**	**0.9472**	**0.7620**
mirrored	full	0.0619	0.1461	0.0235	0.0886
	gt_crop	0.6404	0.9527	0.8030	0.6924
	det_crop	0.6433	0.9662	0.8240	0.6924
mirrored_full	full	0.7004	0.9702	0.9183	0.7582
	gt_crop	0.5267	0.9291	0.5707	0.5772
	det_crop	0.5606	0.9322	0.6470	0.6089

La Figura 3.2 riporta un esempio qualitativo per quattro delle combinazioni modello/modalità della tabella: poiché le differenze tra le predizioni sono di pochi pixel, la figura mostra l'ingrandimento di una singola articolazione.

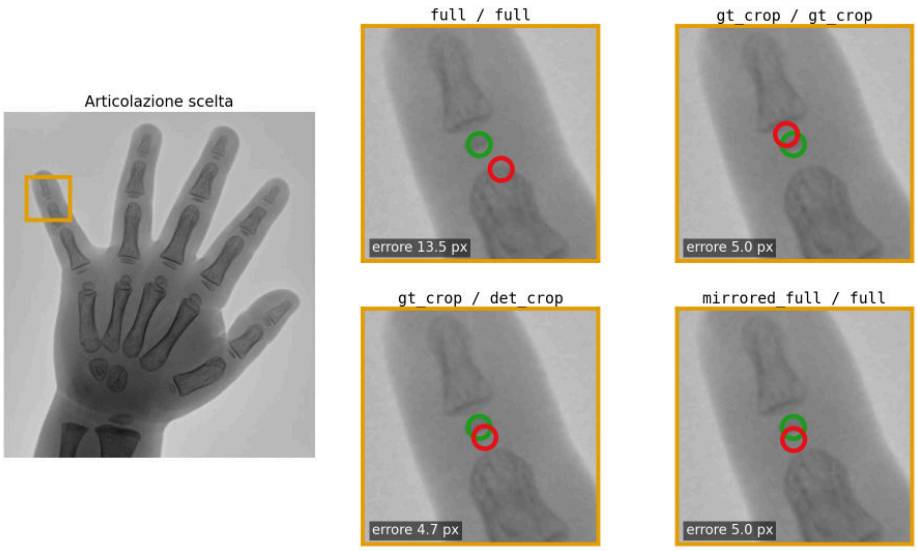


Figura 3.2: Keypoint predetti (rossi) e ground truth (verdi) sulla stessa articolazione, evidenziata nella radiografia a sinistra, con l'errore in pixel di ciascuna predizione, per quattro combinazioni modello/modalità: `full`/`full` (in alto a sinistra), `gt_crop`/`gt_crop` (in alto a destra), `gt_crop`/`det_crop` (in basso a sinistra), `mirrored_full`/`full` (in basso a destra).

Dai risultati emergono tre osservazioni principali.

Innanzitutto, la mAP dipende fortemente dalla coerenza tra dominio di valutazione e dominio di training: i modelli addestrati su crop (`gt_crop`, `mirrored`) ottengono i risultati migliori nelle due modalità basate su crop, quelli addestrati sull'immagine intera (`full`, `mirrored_full`) in modalità `full`. Il caso più estremo è `gt_crop` valutato in modalità `full` (mAP@[.5:.95] = 0.086 sul test set, contro 0.704 nel proprio dominio): il modello non ha mai visto, in training, un target di bounding box diverso dall'intero crop, e di conseguenza non ha mai imparato a localizzare la mano in un contesto più ampio.

In secondo luogo, tra le quattro varianti `mirrored_full` è quella che si comporta in modo più equilibrato tra i domini: mantiene prestazioni ragionevoli anche fuori dal proprio dominio primario (mAP@[.5:.95] $\approx$ 0.53–0.56 su crop, sul test set), contro il quasi totale collasso di `gt_crop` e `mirrored` in modalità `full`. Questo è compatibile con l'ipotesi alla base della sua progettazione (Sez. 2.3.1); i dati disponibili non permettono però di isolare quanto di questo vantaggio derivi dallo specchiamento in sé e quanto dal semplice raddoppio dei dati di training che esso comporta.

Infine, questo comportamento più equilibrato comporta un costo marginale: nel proprio dominio nativo `mirrored_full` raggiunge un mAP@[.5:.95]

di 0.700, quasi allo stesso livello del picco di `gt_crop` (0.704) e nettamente superiore a quello di `mirrored` (0.640) in modalità `gt_crop`. Le varianti allenate su crop restano leggermente più precise quando è disponibile una bounding box di alta qualità a monte, ma il divario si è ridotto a pochi millesimi; `mirrored_full` resta inoltre l'unica in grado di funzionare direttamente sull'immagine intera, gestendo entrambe le mani in un unico passaggio, senza necessitare di una fase di localizzazione separata. Questo confronto riguarda i modelli addestrati sul sottoinsieme: il retraining sul dataset completo, descritto più avanti (Sez. 3.2.2), ne ribalta l'esito a favore di `mirrored_full`.

Confrontando con il paper BoNet [1] (mAP@[.5:.95] = 0.814 con bounding box ground truth sul dataset RHPE, sull'intero training set), i risultati ottenuti su un sottoinsieme di 250 immagini (`gt_crop` in modalità `gt_crop`: 0.704; `mirrored` in modalità `gt_crop`: 0.640) risultano inferiori a quelli riportati da BoNet. La differenza va però interpretata tenendo conto sia della significativa riduzione del dataset di training, sia dell'architettura diversa adottata per la keypoint estimation (Keypoint R-CNN anziché Simple Baselines, Sez. 2.3): non si tratta quindi di un confronto perfettamente controllato. Il ruolo della sola dimensione del dataset viene isolato più avanti in questa sezione, riallenando `gt_crop` sul dataset completo (Sez. 3.2.1).

3.2.1 Retraining di `gt_crop` sul dataset completo

Poiché `gt_crop` è la variante che più fedelmente replica la metodologia di BoNet (Sez. 2.3.1), è stata riallenata sull'intero training set RHPE (5491 immagini, contro le 250 del sottoinsieme stratificato), utilizzando il criterio di validazione basato sulla loss descritto in Sez. 2.3. Una seconda versione è stata inoltre allenata con la data augmentation descritta nella stessa sezione (rotazione e scala casuali). I risultati sul test set sono riportati in Tabella 3.3.

Tabella 3.3: Risultati di `gt_crop` riallenato sul dataset completo, con e senza data augmentation, sul test set. Confronto diretto con le righe `gt_crop` di Tabella 3.2 (sottoinsieme da 250 immagini).

Mode	Augmentation	mAP@[.5:.95]	mAP@.50	mAP@.75	mAR@[.5:.95]
full	no	0.1043	0.2365	0.0847	0.1595
full	sì	0.4017	0.7233	0.4017	0.4570
gt_crop	no	0.7508	0.9587	0.9484	0.7937
gt_crop	sì	0.7699	0.9690	0.9563	0.8013
det_crop	no	0.7330	0.9685	0.9553	0.7899
det_crop	sì	**0.7913**	**0.9741**	**0.9616**	**0.8278**

Rispetto al sottoinsieme da 250 immagini (Tabella 3.2), l'addestramento sul dataset completo migliora le prestazioni ma non in modo decisivo: mAP@[.5:.95] passa da 0.7037 a 0.7508 in modalità `gt_crop` (+0.047) e da 0.7205 a 0.7330 in modalità `det_crop` (+0.013). Il pattern è interessante: mAP@.50 non migliora, mentre mAP@.75 e mAR@[.5:.95] migliorano in entrambe le modalità. Il modello, già con 250 immagini, aveva imparato a localizzare i keypoint approssimativamente nella posizione corretta quasi sempre; il margine di miglioramento con più dati si colloca sulla precisione fine, non sulla capacità di evitare errori grossolani.

L'aggiunta della data augmentation produce un miglioramento ulteriore, non uniforme tra le modalità: marginale nel dominio nativo del modello (`gt_crop`: +0.019), più marcato quando l'input si discosta dalle condizioni di training (`det_crop`: +0.058; `full`: +0.297, quasi quadruplicando il valore di partenza, che resta però basso in termini assoluti). La spiegazione è geometrica: quando la rotazione casuale (Sez. 2.3) ruota il box ground truth, il nuovo box assiale usato per il crop, il più piccolo rettangolo dritto che contiene i quattro angoli ruotati, è per costruzione più grande del box di partenza (tranne che per rotazioni di 0°, 90°, 180°...), aggiungendo quindi margine ulteriore rispetto a quello già presente nell'annotazione originale, quale che sia la sua aderenza alla mano; la componente di scala, al contrario, moltiplica tutte le coordinate per lo stesso fattore e non altera il margine relativo del crop. Il modello si allena quindi, per effetto della sola rotazione, su crop sistematicamente più larghi rispetto all'annotazione di partenza, una condizione più vicina a quella di un box predetto da un detector, in genere meno preciso di un'annotazione manuale: il beneficio dell'augmentation si osserva perciò più chiaramente in modalità `det_crop`, dove questo tipo di margine è già presente indipendentemente dall'augmentation stessa.

Per verificare se applicare l'augmentation a ogni campione (100% dei casi) introducesse una distorsione eccessiva rispetto a un'esposizione più moderata, è stato condotto un ulteriore esperimento su `gt_crop` variando la probabilità di applicazione (Sez. 2.3). I risultati sul test set, in Tabella 3.4, mostrano un effetto reale ma non uniforme tra le modalità: pressoché lineare in modalità `gt_crop`, con rendimenti decrescenti in modalità `det_crop` e `full`, dove la maggior parte del guadagno è già presente al 35%. Il valore al 100% resta comunque il migliore in tutte e tre le modalità. Per l'impiego successivo è stato scelto il modello con probabilità di augmentation del 35%. Applicare le trasformazioni solo a una parte dei campioni limita il rischio di distorsioni eccessive e permette al modello di osservare anche immagini non trasformate. Il run al 100%, pur avendo ottenuto valori più alti sul test set, è mantenuto come termine di confronto.

Tabella 3.4: Effetto della probabilità di applicazione dell'augmentation su `gt_crop`, dataset completo. mAP@[.5:.95] sul test set.

Probabilità	`gt_crop`	`det_crop`	`full`
0%	0.7508	0.7330	0.1043
35%	0.7590	0.7668	0.3527
100%	0.7699	0.7913	0.4017

Con la combinazione dataset completo e augmentation, il divario rispetto al valore riportato da BoNet per RHPE con bounding box ground truth (mAP@[.5:.95] = 0.814) si riduce da 0.110 sul sottoinsieme a 0.044 (`gt_crop` in modalità `gt_crop`: 0.7699); di questa riduzione, 0.019 sono attribuibili alla sola augmentation, che porta il divario da 0.063 a 0.044. Il confronto approfondito con questo valore è discusso nel Capitolo 4.

Le altre due varianti (`full`, `mirrored`) non sono state riallenate sul dataset completo; i valori corrispondenti in Tabella 3.2 restano quindi quelli ottenuti sul sottoinsieme. `mirrored_full` è invece stato riallenato a sua volta, come descritto nella sottosezione seguente.

3.2.2 Retraining di `mirrored_full` sul dataset completo

Oltre a `gt_crop`, anche `mirrored_full` è stato riallenato sull'intero training set RHPE con la stessa data augmentation (rotazione e scala casuali, applicata al 100% dei campioni), per verificare se, con lo stesso trattamento, potesse avvicinarsi alle prestazioni della variante più specializzata pur mantenendo il vantaggio pratico di non richiedere una bounding box esterna. A differenza di `gt_crop`, qui dataset completo e augmentation sono stati introdotti insieme in un unico addestramento, senza un passaggio intermedio senza augmentation: non è quindi possibile isolare separatamente il contributo dei due fattori per questa variante. I risultati sul test set sono riportati in Tabella 3.5, a confronto con quelli sul sottoinsieme (Tabella 3.2).

Tabella 3.5: Risultati di `mirrored_full` riallenato sul dataset completo con data augmentation, confrontati con il sottoinsieme da 250 immagini. mAP@[.5:.95] sul test set.

Mode	Sottoinsieme	Dataset completo + aug	Δ
full	0.7004	0.7909	+0.090
gt_crop	0.5267	0.5368	+0.010
det_crop	0.5606	0.5738	+0.013

Confrontando lo stesso tipo di passaggio per entrambe le varianti (dal

sottoinsieme al dataset completo con augmentation), il miglioramento di `mirrored_full` nel proprio dominio nativo resta più marcato di quello di `gt_crop`: +0.090 (0.7004 → 0.7909, modalità `full`) contro +0.066 (0.7037 → 0.7699, modalità `gt_crop`, Sez. 3.2.1). Non è però possibile, per `mirrored_full`, scomporre questo miglioramento nei contributi separati di più dati e augmentation, a differenza di quanto fatto per `gt_crop`, poiché qui i due fattori sono stati introdotti insieme in un unico addestramento. Fuori dal proprio dominio (modalità `gt_crop` e `det_crop`) il miglioramento resta invece contenuto (+0.010, +0.013), coerentemente con l'idea che il training resti specializzato sul proprio dominio nativo.

Il risultato più rilevante emerge dal confronto con `gt_crop`: il nuovo valore di `mirrored_full` in modalità `full` (0.7909) è praticamente identico al miglior risultato di `gt_crop` in modalità `det_crop` con augmentation (0.7913, Tabella 3.3) e supera quello in modalità `gt_crop` (0.7699). `mirrored_full` raggiunge quindi prestazioni sostanzialmente equivalenti a quelle della variante più specializzata, pur lavorando sull'immagine intera, senza bounding box esterna, e gestendo entrambe le mani in un unico passaggio. Il divario rispetto al valore di riferimento di BoNet (mAP@[.5:.95] = 0.814 con bounding box ground truth) è di 0.023, secondo solo al divario di `gt_crop` in modalità `det_crop` (0.017 contro il valore di BoNet con bbox predetta, Tabella 4.1) tra tutte le combinazioni considerate in questo lavoro; va notato che il confronto non è pienamente omogeneo, poiché `mirrored_full` in modalità `full` non riceve alcuna bounding box in input, né ground truth né predetta, un compito più arduo di entrambe le condizioni riportate da BoNet, il che rende il risultato ancora più significativo.

L'addestramento ha richiesto un giorno e mezzo, significativamente più di quello di `gt_crop`, a causa del raddoppio del dataset dovuto allo specchiamento; non è quindi previsto, per questa variante, lo stesso approfondimento sperimentale condotto per `gt_crop` (confronto isolato con/senza augmentation, ablation sulla probabilità di augmentation).

Per verificare che il miglioramento non riguardi un solo caso, si può considerare l'errore di localizzazione dei keypoint, in pixel dell'immagine originale, su tutte le 79 immagini del test set (17 keypoint ciascuna). Per `gt_crop` in modalità `det_crop` l'errore mediano scende da 7.7 px sul sottoinsieme a 6.7 px con il dataset completo e a circa 6.3–6.4 px con l'augmentation, sia al 35% sia al 100%; per `mirrored_full` scende da 7.5 a 6.3 px. Con l'augmentation si accorcia anche la coda degli errori maggiori (95° percentile da 23.8 a 20.9 px per `gt_crop`, da 23.0 a 19.0 px per `mirrored_full`), coerentemente con l'aumento della mAP. Le due configurazioni di `gt_crop` con augmentation hanno errori mediani praticamente uguali, pur con una mAP più alta al 100% (0.7913 contro 0.7668).

La Figura 3.3 mostra, a titolo di esempio, la stessa articolazione della Fi-

gura 3.2 lungo i passaggi di addestramento: l'errore del keypoint passa da 4.7 px sul sottoinsieme a 2.2 px con dataset completo e augmentation al 100% per `gt_crop` in modalità `det_crop`, e da 5.0 a 1.5 px per `mirrored_full`. L'esempio è scelto perché illustra un miglioramento regolare e non è rappresentativo di tutte le articolazioni: l'evidenza complessiva è quella degli errori su tutto il test set riportati sopra.

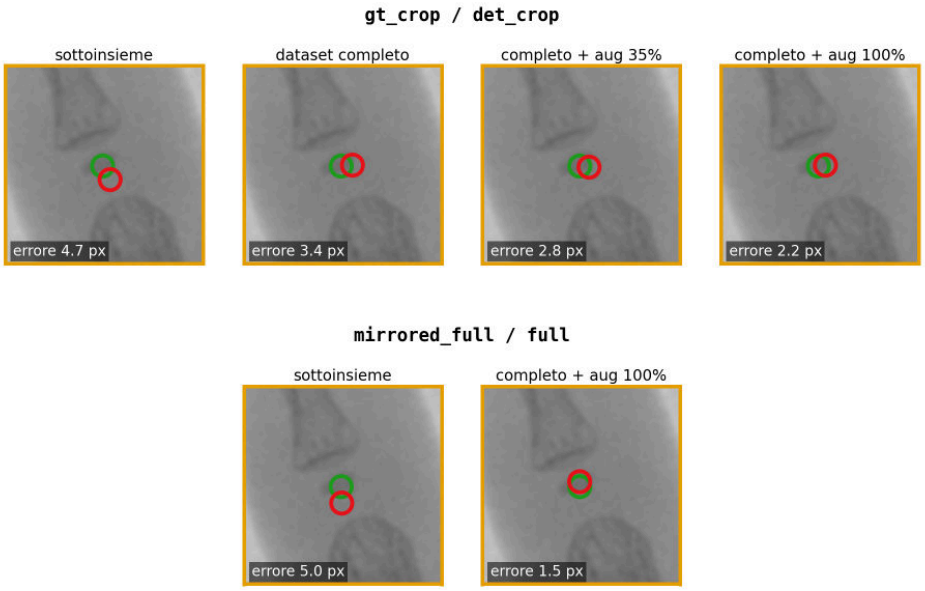


Figura 3.3: Stessa articolazione della Figura 3.2 lungo i passaggi di addestramento: keypoint predetti (rossi) e ground truth (verdi), con l'errore in pixel di ciascuna predizione.

3.3 Esplorazione su entrambe le mani: risultati

L'esplorazione è stata condotta con i modelli addestrati sul sottoinsieme, precedentemente ai retraining descritti in Sez. 3.2.1 e 3.2.2. Il confronto tra le quattro strategie resta quindi omogeneo, poiché tutte impiegano modelli addestrati nelle stesse condizioni; le prestazioni assolute di `mirrored_full` in v4 sono però quelle della versione meno performante, e non riflettono il miglioramento ottenuto con il retraining.

Poiché il test set RHPE non include ground truth per la mano destra, i quattro approcci descritti in Sez. 2.3.5 sono stati valutati qualitativamente, tramite ispezione visiva delle predizioni su un campione di 5 immagini di test:

v1 Il modello `full` non generalizza alla ricerca di una seconda mano, non

avendo mai visto più di una bounding box per immagine in training: le detection aggiuntive ottengono punteggi molto bassi (0.06–0.13) e in alcuni casi corrispondono a doppioni della stessa mano sinistra, non alla destra (Figura 3.4).

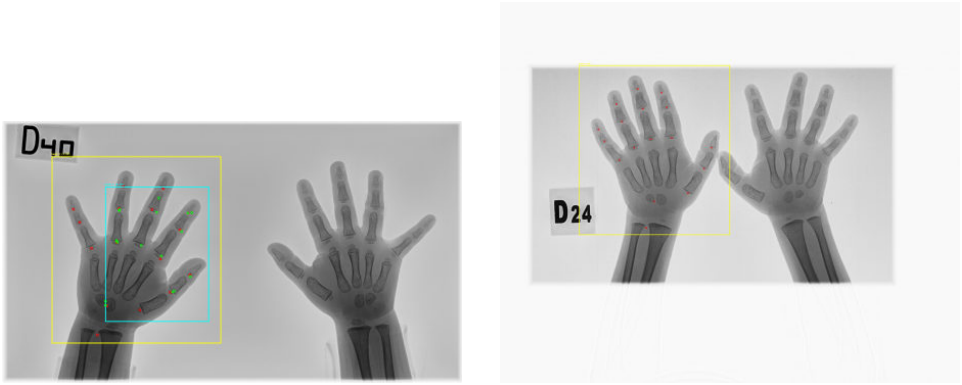


Figura 3.4: v1: detection con `full` sull'immagine intera, keypoint con `mirrored`.

v2 Risultati incostanti: la mano destra viene rilevata con buon punteggio in 2 immagini su 5, con punteggio molto basso (0.09–0.31, sotto soglia) in altre 2, e nessuna detection nella restante (Figura 3.5).

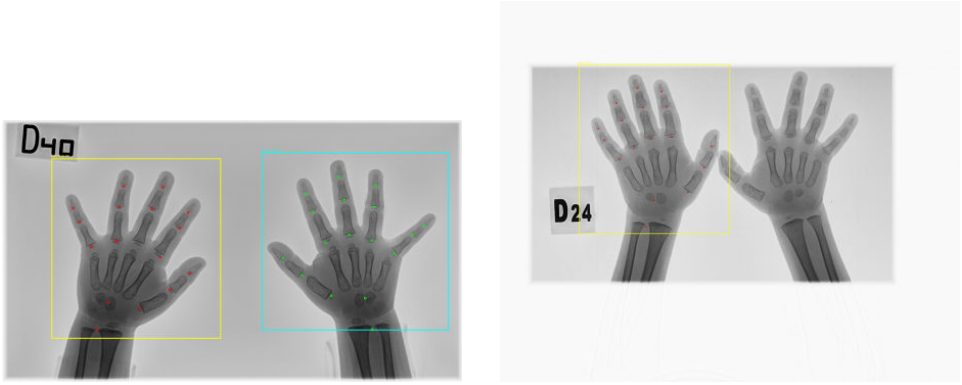


Figura 3.5: v2: immagine divisa a metà, detection con `full` su ciascuna metà, keypoint con `mirrored`.

v3 La detection diventa affidabile su entrambe le metà, con punteggi superiori a 0.98 su tutte e 5 le immagini del campione, ma a un'ispezione visiva i keypoint predetti appaiono errati su entrambe le mani, coerentemente con il limite di generalizzazione di `mirrored` discusso in Sez. 2.3.1 (Figura 3.6).

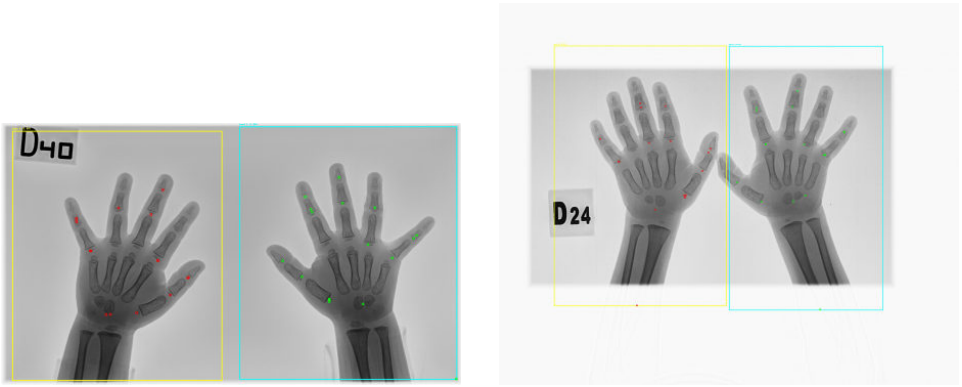


Figura 3.6: v3: immagine divisa a metà, detection e keypoint entrambi con `mirrored`.

v4 Produce direttamente detection e keypoint che, a un'ispezione visiva, appaiono corretti per entrambe le mani, in un unico passaggio: a differenza di v3, qui anche i keypoint appaiono corretti e non solo la detection (Figura 3.7).

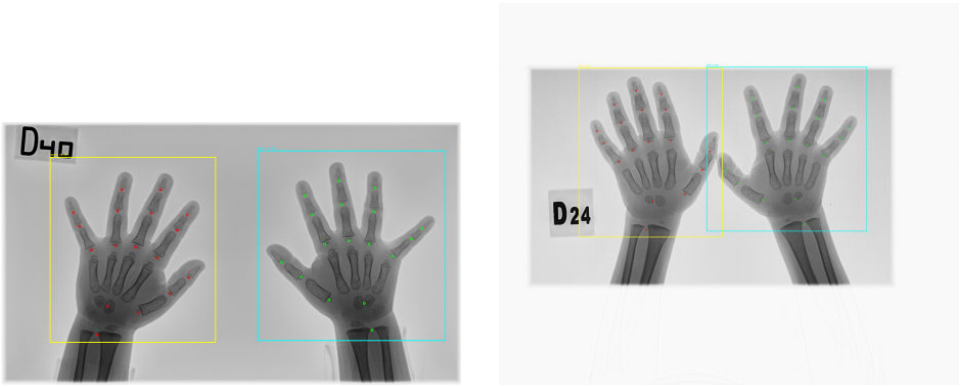


Figura 3.7: v4: singolo forward pass sull'immagine intera con `mirrored_full`.

Tra le quattro strategie esplorate, v4 appare la più efficace ed è coerente con i risultati quantitativi discussi in precedenza; viene quindi adottata come soluzione di riferimento per l'obiettivo esplorativo di predizione su entrambe le mani, tenendo conto che questa valutazione resta qualitativa e basata su un campione ridotto di immagini.

3.4 Sintesi dei risultati

La scelta del modello dipende dallo scenario d'uso. Se serve la sola mano sinistra con una bounding box fornita da un modulo di detection (come nello stadio successivo della pipeline BoNet), la combinazione migliore è `gt_crop` in modalità `det_crop`, con mAP@[.5:.95] = 0.7913. In un contesto clinico, invece, l'immagine può mostrare solo la mano sinistra, solo la destra o entrambe: `mirrored_full` è l'unica variante addestrata su entrambe le orientazioni che opera sull'immagine intera senza bounding box esterna, e raggiunge sulla sinistra una mAP sostanzialmente equivalente (0.7909). Per questo la si considera più adatta a gestire la variabilità dei casi clinici, anche se non è il miglior risultato assoluto. Va precisato che la sua capacità di gestire la mano destra è verificata solo qualitativamente (Sez. 3.3).

Capitolo 4

Discussione

4.1 Confronto con il paper di riferimento

Il paper BoNet [1] riporta, per la keypoint estimation sul dataset RHPE, un mAP@[.5:.95] di 0.814 con bounding box ground truth e di 0.808 con bounding box predetta. La Tabella 4.1 confronta questi valori con i risultati ottenuti in questo lavoro, sia sul sottoinsieme da 250 immagini sia con `gt_crop` e `mirrored_full` riallenati sul dataset completo con data augmentation (Sez. 3.2.1, 3.2.2).

Tabella 4.1: Confronto tra i valori di BoNet e i risultati di questo lavoro, sul sottoinsieme da 250 immagini e sul dataset completo con data augmentation.

Condizione	mAP@[.5:.95]	Divario
Modello `gt_crop`, bounding box ground truth		
BoNet	0.814	
Sottoinsieme	0.7037	0.110
Dataset completo + aug	0.7699	0.044
Modello `gt_crop`, bounding box predetta (`det_crop`)		
BoNet	0.808	
Sottoinsieme	0.7205	0.088
Dataset completo + aug	0.7913	0.017
Modello `mirrored_full`, nessuna bounding box in input		
BoNet (bbox ground truth)	0.814	
Sottoinsieme	0.7004	0.114
Dataset completo + aug	0.7909	0.023

La Figura 4.1 mostra lo stesso confronto lungo i passaggi di riaddestramento: per ciascun modello, il valore sul sottoinsieme da 250 immagini,

quello con il dataset completo e con augmentation al 35% e al 100% (dove disponibili), e il valore di riferimento di BoNet.

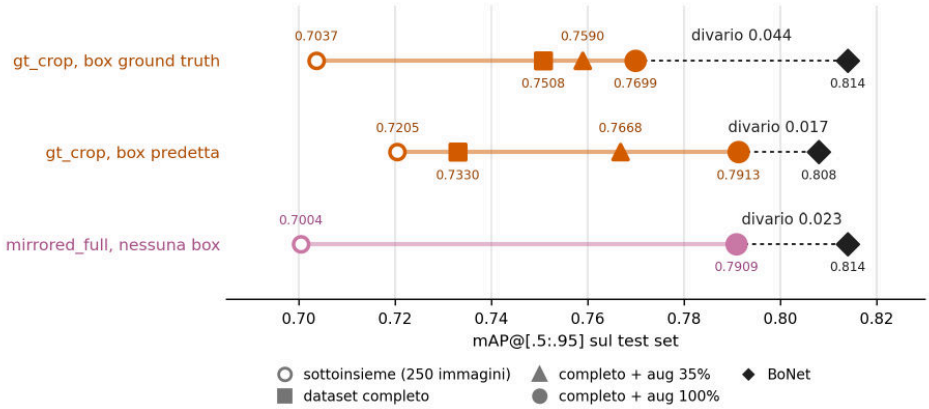


Figura 4.1: Andamento della mAP@[.5:.95] sul test set lungo i riaddestramenti, confrontato con BoNet. Il valore di BoNet è quello con bounding box ground truth (0.814), tranne per la seconda riga, dove è quello con bounding box predetta (0.808); il divario è calcolato rispetto all'ultimo passaggio. Per `mirrored_full` esiste solo la versione con augmentation al 100%. L'asse parte da 0.685.

Il valore in modalità `det_crop` supera quello in modalità `gt_crop` per un motivo già discusso in Sez. 3.2.1 (l'augmentation avvantaggia in modo particolare gli input con un margine maggiore attorno alla mano), non per un effetto isolato o casuale. Il divario più piccolo in assoluto resta quello di `gt_crop`/`det_crop` (0.017); `mirrored_full` si avvicina quasi altrettanto (0.023) pur non ricevendo alcuna bounding box in input, un confronto quindi non pienamente omogeneo con le altre righe della tabella (Sez. 3.2.2). Interpretare questi divari richiede comunque di considerare almeno due differenze non isolate in questo lavoro.

La prima è architetturale: BoNet utilizza Simple Baselines for Human Pose Estimation [16], mentre questo lavoro utilizza Keypoint R-CNN [6]. Un divario di prestazioni tra le due architetture è documentato indipendentemente sul benchmark standard della pose estimation, il task di keypoint detection del dataset COCO [7] (un compito diverso, la posa umana su fotografie anziché mani radiografiche, ma quello su cui entrambi i paper riportano i propri risultati): Simple Baselines con backbone ResNet-50 raggiunge un AP di 70.4 su COCO val2017 [16], mentre Mask R-CNN con lo stesso backbone (ResNet-50-FPN), da cui deriva Keypoint R-CNN, raggiunge un AP di 62.7 su COCO test-dev [6] (le due valutazioni usano split diversi del dataset, quindi il confronto numerico esatto va preso con cautela, ma la

direzione e l'ordine di grandezza del divario, 7-8 punti, sono coerenti con quanto osservato in questo lavoro). La spiegazione tipicamente proposta è che Simple Baselines opera su un crop già ritagliato ad alta risoluzione con un decoder a deconvoluzione dedicato, mentre Keypoint R-CNN predice i keypoint da una feature map ottenuta tramite RoIAlign, storicamente meno precisa per la localizzazione fine.

La seconda è una discrepanza nella ricetta di training, mai isolata prima d'ora. Il paper BoNet indica, per la keypoint estimation, solo il numero di epoche (20) e l'uso dei "parametri suggeriti" da Simple Baselines: ottimizzatore Adam, learning rate 10^{-3}, batch size 128. Poiché Keypoint R-CNN appartiene alla famiglia Faster R-CNN, tradizionalmente addestrata con SGD, questo lavoro ha ereditato da BoNet solo il numero di epoche, non il resto della ricetta: l'addestramento usa SGD con batch size 4, un numero di aggiornamenti dei pesi per epoca circa 32 volte più alto a parità di epoche nominali. "20 epoche" non rappresenta quindi lo stesso volume di addestramento nei due casi.

L'introduzione della data augmentation descritta in Sez. 2.3, motivata proprio da questo confronto, ha permesso di ridurre il divario da 0.063 a 0.044 (modalità `gt_crop`) senza modificare l'architettura né la ricetta di ottimizzazione. Questo è compatibile con l'idea che il divario residuo derivi da una combinazione delle due differenze appena descritte, sebbene isolarne il contributo relativo richiederebbe esperimenti dedicati, che restano fuori dallo scope di questo lavoro.

4.2 Osservazioni metodologiche

La Figura 4.2 riassume visivamente i risultati di Tabella 3.2 sul test set, alla base delle osservazioni che seguono.

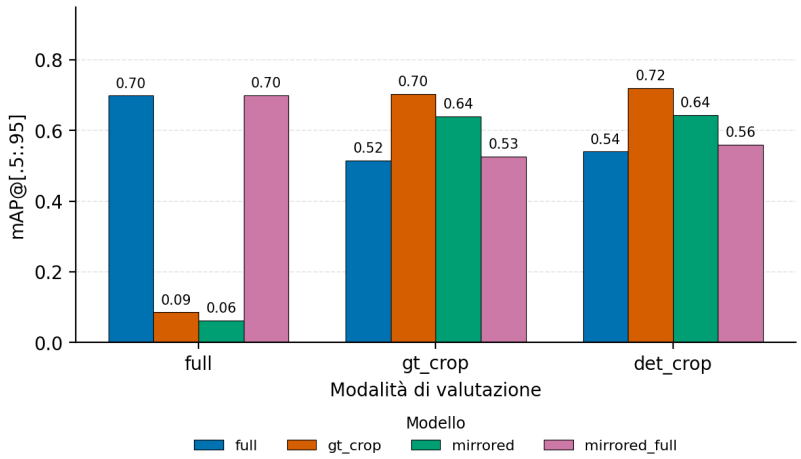


Figura 4.2: mAP@[.5:.95] sul test set per le quattro varianti di modello nelle tre modalità di valutazione (Tabella 3.2). I modelli addestrati su crop (`gt_crop`, `mirrored`) crollano in modalità `full`; i modelli addestrati sull'immagine intera (`full`, `mirrored_full`) restano invece ragionevoli in tutte e tre le modalità.

Il pattern osservato in Sez. 3.2, un crollo quasi totale delle prestazioni quando un modello viene valutato fuori dal dominio visto in training, non è un artefatto specifico di questo lavoro, ma un'istanza di un fenomeno più generale: un modello di apprendimento supervisionato non impara un compito in astratto, impara a risolverlo assumendo implicitamente che i dati futuri seguano la stessa distribuzione di quelli visti in addestramento (Sez. 1.1.1). Più dati e data augmentation attenuano parzialmente questo crollo, senza eliminarlo: lo stesso `gt_crop` valutato in modalità `full`, riallenato sul dataset completo con augmentation, passa da 0.086 a 0.40 (Sez. 3.2.1), un miglioramento reale ma ancora lontano dal picco del modello nel proprio dominio (0.77).

Nel caso di `gt_crop` e `mirrored`, questa assunzione implicita è particolarmente rigida: il target di bounding box è sempre l'intero crop, quindi il modello non solo non ha mai visto immagini con sfondo, ma non ha mai avuto un incentivo ad apprendere a distinguere la mano dal contesto circostante.

Un confronto più specifico è compatibile con questa lettura. `full` e `mirrored_full` sono gli unici due modelli addestrati sull'immagine intera; quando entrambi vengono valutati fuori da questo dominio, su crop, `mirrored_full` supera `full` in modo sistematico: sul test set, mAP@[.5:.95] in modalità `gt_crop` 0.527 contro 0.515 e in modalità `det_crop` 0.561 contro 0.541 (Tabella 3.2), con lo stesso ordinamento riscontrato anche sugli split di training e validation, non riportati in tabella. I risultati sono compatibili con l'ipotesi che il raddoppio dei dati di training introdotto dallo specchia-

mento, pensato per predire keypoint su entrambe le mani, abbia anche un effetto positivo secondario sulla capacità di generalizzazione del modello; non è però possibile, con gli esperimenti disponibili, isolare questo effetto da spiegazioni alternative (ad esempio un semplice aumento della quantità di dati, indipendente dallo specchiamento in sé), poiché non è stato addestrato un modello di controllo su 500 immagini non specchiate.

Questo suggerisce una lettura pratica oltre a quella puramente numerica: sebbene Keypoint R-CNN sia di per sé un'architettura integrata capace di detection e keypoint estimation insieme, la testa di detection di `gt_crop` e `mirrored` non ha mai imparato a localizzare la mano in un contesto ampio, essendo stata addestrata con un target di bounding box sempre coincidente con l'intero crop. In pratica, questi due modelli restano quindi utilizzabili solo se a monte è disponibile una bounding box prodotta altrove (dal ground truth o da un sistema di detection dedicato, come nella modalità `det_crop`): non è un limite architetturale di Keypoint R-CNN, ma una conseguenza diretta di come sono stati addestrati. `full` e `mirrored_full`, addestrati invece sull'intera immagine, non hanno questo vincolo e funzionano in autonomia end-to-end; tra i due, solo `mirrored_full` lo fa su entrambe le mani (Sez. 3.3). Già sul sottoinsieme nessuna delle varianti allenate su crop lo supera in modo netto nel confronto di picco (`gt_crop`: 0.704, sostanzialmente alla pari; `mirrored`: 0.640, visibilmente indietro, Sez. 3.2), e dopo il retraining sul dataset completo il confronto si ribalta a suo favore (0.7909 contro 0.7699 di `gt_crop` nel rispettivo dominio nativo, Sez. 3.2.2).

4.3 Limiti del lavoro

Un primo limite riguarda la mano destra: le quattro strategie confrontate in Sez. 3.3 sono state valutate solo tramite ispezione visiva su un campione ridotto di immagini, poiché il dataset RHPE annota esclusivamente la mano sinistra. Non è quindi possibile fornire una misura quantitativa dell'accuratezza su questa mano, né confrontarla direttamente con le prestazioni ottenute sulla sinistra.

In secondo luogo, l'intera estensione alla mano destra si fonda sull'assunzione che le due mani siano approssimabili tramite uno specchiamento orizzontale (Sez. 2.3.4). Si tratta di una semplificazione geometrica: non tiene conto di eventuali asimmetrie reali tra le due mani, né di differenze di rotazione o posizionamento rispetto all'asse di specchiamento, e la sua validità non è stata verificata clinicamente o anatomicamente.

Un terzo limite riguarda la dimensione del test set, composto da 79 immagini (Tabella 2.1). Su un campione di queste dimensioni, differenze di pochi millesimi di mAP, come quelle che separano alcune delle varianti

confrontate nel Capitolo 3, sono difficilmente distinguibili dalla variabilità campionaria: i confronti tra valori molto ravvicinati vanno quindi letti come indicazioni qualitative, non come ordinamenti stabili. Non sono stati calcolati intervalli di confidenza, né ripetuti gli addestramenti con inizializzazioni diverse, cosa che avrebbe permesso di quantificare questa incertezza.

Infine, tutti i modelli sono stati addestrati e valutati su un'unica fonte di dati, il dataset RHPE, raccolto da una singola coorte con un protocollo di acquisizione specifico. Si tratta, nella sostanza, dello stesso limite discusso per BoneXpert in Sez. 1.2: cambiare la popolazione di riferimento, da danese a colombiana, non elimina il problema strutturale di validare un modello su un'unica coorte. Il comportamento dei modelli proposti in questo lavoro su dati provenienti da popolazioni, protocolli di acquisizione o apparecchiature diverse resta quindi non verificato.

Capitolo 5

Conclusioni e sviluppi futuri

5.1 Riepilogo dei contributi

La replica dei primi due stadi della pipeline BoNet [1], hand detection e keypoint estimation, è stata realizzata adattando al dataset RHPE i modelli di torchvision inizializzati con pesi pre-addestrati su COCO, in assenza di checkpoint forniti dagli autori. Le prestazioni raggiunte sono comparabili a quelle del paper di riferimento (Capitolo 3).

Il secondo obiettivo, la predizione dei keypoint su entrambe le mani, ha portato al confronto di quattro varianti di modello e quattro strategie di combinazione (Sez. 2.3.5), da cui è emersa `mirrored_full` come la variante più promettente tra quelle esplorate: l'unica in grado di produrre detection e keypoint per entrambe le mani in un unico passaggio, senza necessitare di una bounding box esterna (Sez. 3.3, Capitolo 4). Riallenata sul dataset completo con data augmentation, raggiunge un mAP@[.5:.95] di 0.7909, sostanzialmente equivalente al miglior risultato ottenuto da `gt_crop` (0.7913) pur senza richiedere alcuna bounding box in input, e a soli 0.023 dal valore di riferimento di BoNet (Sez. 3.2.2).

5.2 Sviluppi futuri

I limiti discussi in Sez. 4.3 indicano alcune direzioni naturali per estendere il lavoro.

La prima riguarda la generalizzazione. Il dataset RHPE proviene da un unico centro, la Fundación Santa Fe de Bogotá, e i risultati ottenuti non dicono nulla su immagini acquisite con altre apparecchiature, protocolli o popolazioni di pazienti. Una direzione è quindi la validazione su più centri, per verificare se le prestazioni si mantengono; un'altra è l'addestramento su

un dataset multicentrico, che esporrebbe i modelli a una variabilità maggiore e potrebbe renderli più robusti.

Un'ulteriore direzione riguarda il modello di partenza. I modelli utilizzati sono adattati tramite fine-tuning a partire da pesi pre-addestrati su COCO, un dataset di fotografie di scene naturali (nel caso dei keypoint, per la stima della posa di persone): nessuna radiografia è vista prima del fine-tuning su RHPE. Si potrebbe partire invece da un modello di base pre-addestrato su una quantità di immagini di ordini di grandezza superiore, oppure su immagini mediche, per verificare se ne derivi un miglioramento, in particolare quando le immagini annotate sono poche.

Resta infine da completare la validazione dell'estensione alla mano destra, valutata qui solo qualitativamente per l'assenza di annotazioni in RHPE. Annotare un campione di immagini permetterebbe di misurare le prestazioni di `mirrored_full` e di isolare il contributo dello specchiamento da quello del dataset completo e della data augmentation, che nel retraining sono stati variati insieme.

Bibliografia

[1] María Escobar et al. «Hand Pose Estimation for Pediatric Bone Age Assessment». In: *Medical Image Computing and Computer Assisted Intervention – MICCAI 2019*. Vol. 11769. Lecture Notes in Computer Science. 2019, pp. 531–539. DOI: 10.1007/978-3-030-32226-7_59.

[2] Ian Goodfellow, Yoshua Bengio e Aaron Courville. *Deep Learning*. MIT Press, 2016.

[3] William Walter Greulich e S. Idell Pyle. *Radiographic Atlas of Skeletal Development of the Hand and Wrist*. 2ª ed. Stanford University Press, 1959.

[4] Safwan S. Halabi, Luciano M. Prevedello et al. «The RSNA Pediatric Bone Age Machine Learning Challenge». In: *Radiology* 290.2 (2019), pp. 498–503. DOI: 10.1148/radiol.2018180736.

[5] Kaiming He et al. «Deep Residual Learning for Image Recognition». In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016.

[6] Kaiming He et al. «Mask R-CNN». In: *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*. 2017.

[7] Tsung-Yi Lin, Michael Maire, Serge Belongie et al. «Microsoft COCO: Common Objects in Context». In: *Computer Vision – ECCV 2014*. 2014.

[8] Tsung-Yi Lin et al. «Feature Pyramid Networks for Object Detection». In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2017.

[9] Vinod Nair e Geoffrey E. Hinton. «Rectified Linear Units Improve Restricted Boltzmann Machines». In: *Proceedings of the 27th International Conference on Machine Learning (ICML)*. 2010, pp. 807–814.

[10] Shaoqing Ren et al. «Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks». In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2015.

[11] Matteo Ruggero Ronchi e Pietro Perona. «Benchmarking and Error Diagnosis in Multi-Instance Pose Estimation». In: *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*. 2017.

[12] David E. Rumelhart, Geoffrey E. Hinton e Ronald J. Williams. «Learning representations by back-propagating errors». In: *Nature* 323.6088 (1986), pp. 533–536. DOI: 10.1038/323533a0.

[13] Ilya Sutskever et al. «On the importance of initialization and momentum in deep learning». In: *Proceedings of the 30th International Conference on Machine Learning (ICML)*. 2013.

[14] James M. Tanner et al. *Assessment of Skeletal Maturity and Prediction of Adult Height (TW2 Method)*. London: Academic Press, 1975.

[15] Hans Henrik Thodberg et al. «The BoneXpert Method for Automated Determination of Skeletal Maturity». In: *IEEE Transactions on Medical Imaging* 28.1 (2009), pp. 52–66. DOI: 10.1109/TMI.2008.926067.

[16] Bin Xiao, Haiping Wu e Yichen Wei. «Simple Baselines for Human Pose Estimation and Tracking». In: *Computer Vision – ECCV 2018*. 2018. DOI: 10.1007/978-3-030-01231-1_29.